

VIT®
CHENNAI
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

"Predicting Credit Card Approval and Auditing Fairness Across Applicant Subgroups"

By

Ayush Khatai(24BDS1106)

Submitted to

Prof. Jaganathan B

BMAT202P Probability and statistics

In

BTECH Computer Science and Engineering

1. Problem Statement and Objectives

Financial institutions rely heavily on automated credit scoring systems to decide whether to approve or reject credit card applications. While machine learning models can improve the speed and consistency of these decisions, they can also inherit or amplify biases present in historical data, leading to systematically unfavourable outcomes for certain applicant subgroups even when protected attributes are not explicitly used.

This project has two objectives. First, to build and compare multiple machine learning models that predict credit card approval decisions from applicant attributes. Second, to evaluate whether the resulting predictions are distributed fairly across encoded applicant subgroups, treating this as a lightweight fairness audit rather than a full causal fairness study given the anonymized nature of the data.

Objectives

- Build a clean, well-understood dataset from the anonymized UCI Credit Approval data.
- Engineer features and connect the processed dataset to a relational/NoSQL database for retrieval-based analysis .
- Train and compare 10-15 classification algorithms, including at least one deep learning model .
- Evaluate models on standard performance metrics and on a basic group-fairness metric across an encoded sensitive-looking attribute.
- Present results through an interactive dashboard and a final report with limitations and future scope .

2. Dataset Identification and Justification

Dataset: UCI Machine Learning Repository - Credit Approval Dataset (id: 27), Quinlan, J. (1987), DOI: 10.24432/C5FS30

The UCI Credit Approval dataset contains 690 real credit card applications with 15 attributes and a binary approval outcome. All attribute names and values have been anonymized to protect confidentiality, but the dataset retains a realistic mix of continuous, low-cardinality categorical, and high-cardinality categorical features, along with a small number of missing values in two attributes.

This dataset is well suited to the assignment for three reasons. It requires genuine preprocessing decisions because of its missing values and mixed data types, unlike fully clean toy datasets. Its small size (690 rows) keeps training and comparison of 10-15 models computationally practical within the course timeline. Finally, because the attributes are anonymized, the dataset is an appropriate and low-risk setting to practice fairness-auditing techniques without exposing any real individual's protected information, while still connecting to a highly active area of applied machine learning research in banking and lending.

3. Literature Review

1. Le Quy, T., et al. (2024). An experimental study on fairness-aware machine learning for credit scoring problem. arXiv:2412.20298.

Benchmarks several fairness-aware learning methods against standard classifiers on widely used credit datasets and evaluates them using multiple fairness measures, finding that fairness interventions carry varying trade-offs against predictive accuracy depending on the dataset and metric used.

2. Kozodoi, N., Jacob, J., & Lessmann, S. (2022). Fairness in credit scoring: Assessment, implementation and profit implications. European Journal of Operational Research, 297(3), 1083-1094.

Compares independence, separation, and sufficiency as fairness criteria for credit scoring and recommends separation as the more appropriate standard because it accounts for differing misclassification costs across applicant groups.

3. Moscato, V., Picariello, A., & Sperli, G. (2021). A benchmark of machine learning approaches for credit score prediction. Expert Systems with Applications, 165, 113986.

Systematically benchmarks a wide range of classical and ensemble machine learning algorithms for credit score prediction, providing a useful reference point for algorithm selection and comparison methodology.

4. Das, S., Stanton, R., & Wallace, N. (2023). Algorithmic Fairness. Annual Review of Financial Economics, 15, 565-593.

Reviews the financial-economics literature on algorithmic fairness in lending, contrasting human versus machine sources of bias and surveying group- versus individual-level fairness metrics, and notes that modern ML models tend to outperform logistic regression at the cost of interpretability.

5. Fairness in focus: quantitative insights into bias within machine learning risk evaluations and established credit models. (2025). Management System Engineering (Springer Nature).

Proposes a quantitative heuristic for auditing bias across an expanded set of protected categories in credit risk models, extending prior fairness benchmarks beyond the commonly studied race and gender attributes.

4. Dataset Understanding and Feature Description

The dataset has 690 instances and 16 columns (15 features, A1-A15, plus target A16).

Attribute	Coded Values	Type	Working Interpretation (proxy)
A1	b, a	Categorical	Applicant gender (proxy)
A2	continuous	Continuous	Applicant age (proxy)
A3	continuous	Continuous	Debt / existing liability amount (proxy)
A4	u, y, l, t	Categorical	Marital status (proxy)
A5	g, p, gg	Categorical	Bank customer type (proxy)
A6	c, d, cc, i, j, k, m, r, q, w, x, e, aa, ff	Categorical	Education / occupation level (proxy)
A7	v, h, bb, j, n, z, dd, ff, o	Categorical	Ethnicity (proxy, anonymized)
A8	continuous	Continuous	Years employed (proxy)
A9	t, f	Categorical	Prior default history (proxy)
A10	t, f	Categorical	Employment status (proxy)
A11	continuous	Continuous	Credit score (proxy)
A12	t, f	Categorical	Has driver's license (proxy)
A13	g, p, s	Categorical	Citizenship status (proxy)
A14	continuous	Continuous	Zip code (proxy)
A15	continuous	Continuous	Income (proxy)
A16	+, -	Categorical (Target)	Approval decision (approved / rejected)

A7 and A14 contain the dataset's missing values, marked as "?" in the raw file, and will be imputed during preprocessing rather than dropped, to preserve sample size given the dataset is already small.

5. Data Preprocessing

- Replace "?" placeholders with proper NA values on import.
- Impute missing continuous values (A2, A14) using median imputation; impute missing categorical values (A1, A7) using mode imputation.
- Encode low-cardinality categorical attributes (A1, A9, A10, A12) as binary/factor variables.
- Encode higher-cardinality categorical attributes (A4, A5, A6, A7, A13) using one-hot or target encoding and evaluating .
- Scale/normalize continuous attributes (A2, A3, A8, A11, A14, A15) using min-max or z-score normalization.
- Check and treat outliers in continuous attributes using IQR-based capping where appropriate.
- Recode the target A16 (+/-) into a binary 1/0 label for modelling.

Code:

```
> # ============================================================
> # DA1 - Exploratory Data Analysis
> # Dataset: UCI Credit Approval (crx.data)
> # ============================================================
>
> # ---- 0. Setup ----
> # install.packages(c("tidyverse", "naniar", "corrplot"))  # run once if needed
> library(tidyverse)
── Attaching core tidyverse packages ──────────────────────────────────────────────────────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.2.1     ✓ readr     2.2.0
✓ forcats   1.0.1     ✓ stringr   1.6.0
✓ ggplot2   4.0.3     ✓ tibble    3.3.1
✓ lubridate 1.9.5     ✓ tidyr     1.3.2
✓ purrr     1.2.2
── Conflicts ──────────────────────────────────────────────────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package to force all conflicts to become errors
> library(naniar)     # for missing value visualization
> library(corrplot)   # for correlation heatmap

corrplot 0.95 loaded

>
> # ---- 1. Load data ----
> # crx.data has NO header row and uses "?" for missing values
> col_names <- paste0("A", 1:16)
>
> credit <- read.csv(
+   "crx.data",
+   header = FALSE,
+   na.strings = "?",
+   col.names = col_names,
+   stringsAsFactors = FALSE
+ )
>
> str(credit)
```

```
> # ---- 2. Basic cleaning ----
> # Recode target A16 (+/-) into readable factor and binary label
> credit <- credit %>%
+   mutate(
+     A16 = factor(A16, levels = c("+", "-"), labels = c("Approved", "Rejected")),
+     approved_flag = ifelse(A16 == "Approved", 1, 0)
+   )
>
> # Convert categorical columns to factors
> cat_cols <- c("A1", "A4", "A5", "A6", "A7", "A9", "A10", "A12", "A13")
> credit[cat_cols] <- lapply(credit[cat_cols], as.factor)
>
> # Confirm numeric columns are numeric
> num_cols <- c("A2", "A3", "A8", "A11", "A14", "A15")
> credit[num_cols] <- lapply(credit[num_cols], as.numeric)
>
> # Check missing values
> colSums(is.na(credit))
          A1           A2           A3           A4           A5           A6           A7           A8
          12           12            0            6            6            9            9            0
          A9          A10          A11          A12          A13          A14          A15          A16
           0            0            0            0            0           13            0            0
approved_flag
           0
>
> # ---- 3. Handle missing values (used for EDA only; final imputation in DA2) ----
> credit_clean <- credit %>%
+   mutate(
+     A2  = ifelse(is.na(A2),  median(A2,  na.rm = TRUE), A2),
+     A14 = ifelse(is.na(A14), median(A14, na.rm = TRUE), A14)
+   )
> # A1 and A7 (categorical) missing values imputed with mode
> mode_impute <- function(x) {
+   ux <- na.omit(x)
+   ux[which.max(tabulate(match(x, ux)))]
+ }
> credit_clean$A1 <- ifelse(is.na(credit_clean$A1), mode_impute(credit_clean$A1), as.character(credit_clean$A1))
> credit_clean$A7 <- ifelse(is.na(credit_clean$A7), mode_impute(credit_clean$A7), as.character(credit_clean$A7))
> credit_clean$A1 <- as.factor(credit_clean$A1)
> credit_clean$A7 <- as.factor(credit_clean$A7)
>
'data.frame':   690 obs. of  16 variables:
 $ A1 : chr  "b" "a" "a" "b" ...
 $ A2 : num  30.8 58.7 24.5 27.8 20.2 ...
 $ A3 : num  0 4.46 0.5 1.54 5.62 ...
 $ A4 : chr  "u" "u" "u" "u" ...
 $ A5 : chr  "g" "g" "g" "g" ...
 $ A6 : chr  "w" "q" "q" "w" ...
 $ A7 : chr  "v" "h" "h" "v" ...
 $ A8 : num  1.25 3.04 1.5 3.75 1.71 ...
 $ A9 : chr  "t" "t" "t" "t" ...
 $ A10: chr  "t" "t" "f" "t" ...
 $ A11: int  1 6 0 5 0 0 0 0 0 0 ...
 $ A12: chr  "f" "f" "f" "t" ...
 $ A13: chr  "g" "g" "g" "g" ...
 $ A14: int  202 43 280 100 120 360 164 80 180 52 ...
 $ A15: int  0 560 824 3 0 0 31285 1349 314 1442 ...
 $ A16: chr  "+" "+" "+" "+" ...
> summary(credit)
         A1           A2               A3                A4           A5           A6           A7
 Length   :690   Min.   :13.75   Min.   : 0.000   Length   :690   Length   :690   Length   :690   Length   :690
 N.unique :  2   1st Qu.:22.60   1st Qu.: 1.000   N.unique :  3   N.unique :  3   N.unique : 14   N.unique :  9
 N.blank  :  0   Median :28.46   Median : 2.750   N.blank  :  0   N.blank  :  0   N.blank  :  0   N.blank  :  0
 Min.nchar:  1   Mean   :31.57   Mean   : 4.759   Min.nchar:  1   Min.nchar:  1   Min.nchar:  1   Min.nchar:  1
 Max.nchar:  1   3rd Qu.:38.23   3rd Qu.: 7.207   Max.nchar:  1   Max.nchar:  2   Max.nchar:  2   Max.nchar:  2
 NAs      : 12   Max.   :80.25   Max.   :28.000   NAs      :  6   NAs      :  6   NAs      :  9   NAs      :  9
                 NAs    :12
       A8               A9           A10           A11              A12           A13           A14
 Min.   : 0.000   Length   :690   Length   :690   Min.   : 0.0   Length   :690   Length   :690   Min.   :   0
 1st Qu.: 0.165   N.unique :  2   N.unique :  2   1st Qu.: 0.0   N.unique :  2   N.unique :  3   1st Qu.:  75
 Median : 1.000   N.blank  :  0   N.blank  :  0   Median : 0.0   N.blank  :  0   N.blank  :  0   Median : 160
 Mean   : 2.223   Min.nchar:  1   Min.nchar:  1   Mean   : 2.4   Min.nchar:  1   Min.nchar:  1   Mean   : 184
 3rd Qu.: 2.625   Max.nchar:  1   Max.nchar:  1   3rd Qu.: 3.0   Max.nchar:  1   Max.nchar:  1   3rd Qu.: 276
 Max.   :28.500                                   Max.   :67.0                                   Max.   :2000
                                                                                                 NAs    :13
      A15               A16
 Min.   :     0.0   Length   :690
 1st Qu.:     0.0   N.unique :  2
 Median :     5.0   N.blank  :  0
 Mean   :  1017.4   Min.nchar:  1
 3rd Qu.:   395.5   Max.nchar:  1
 Max.   :100000.0
```

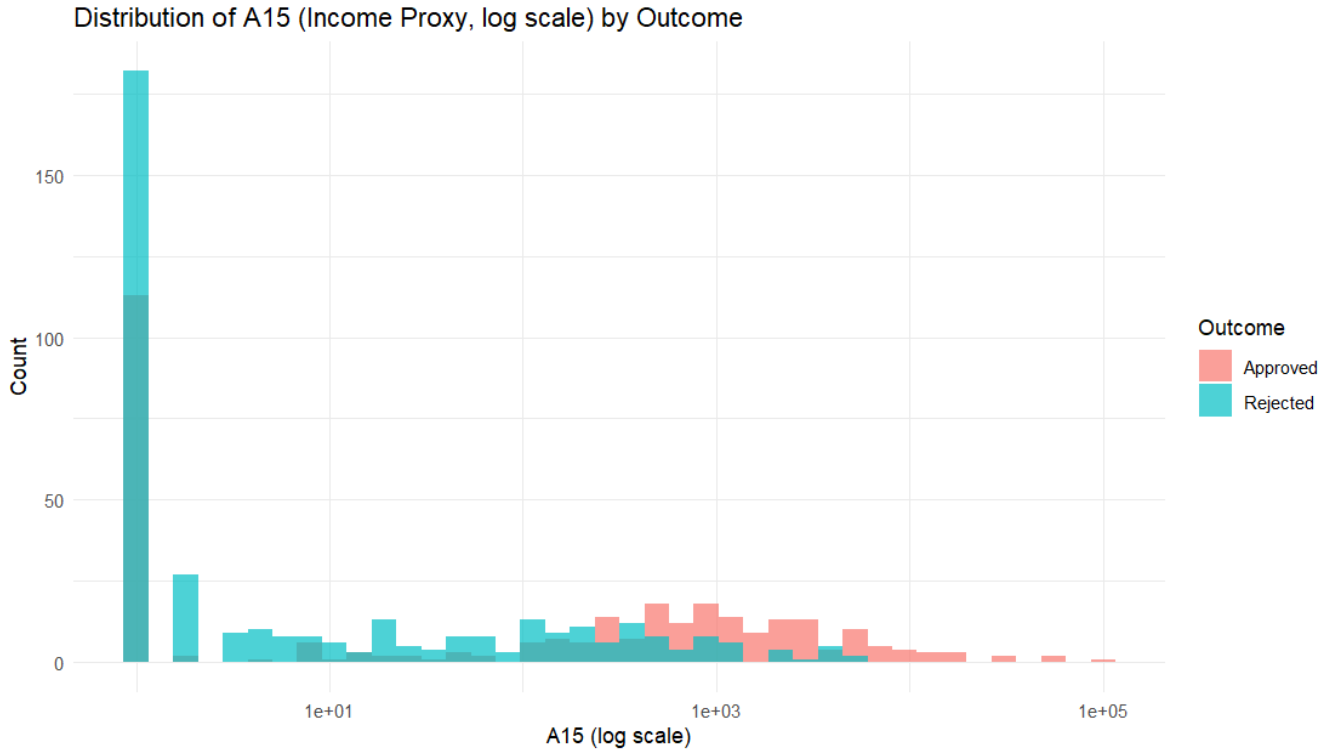
6. Exploratory Data Analysis

Code:

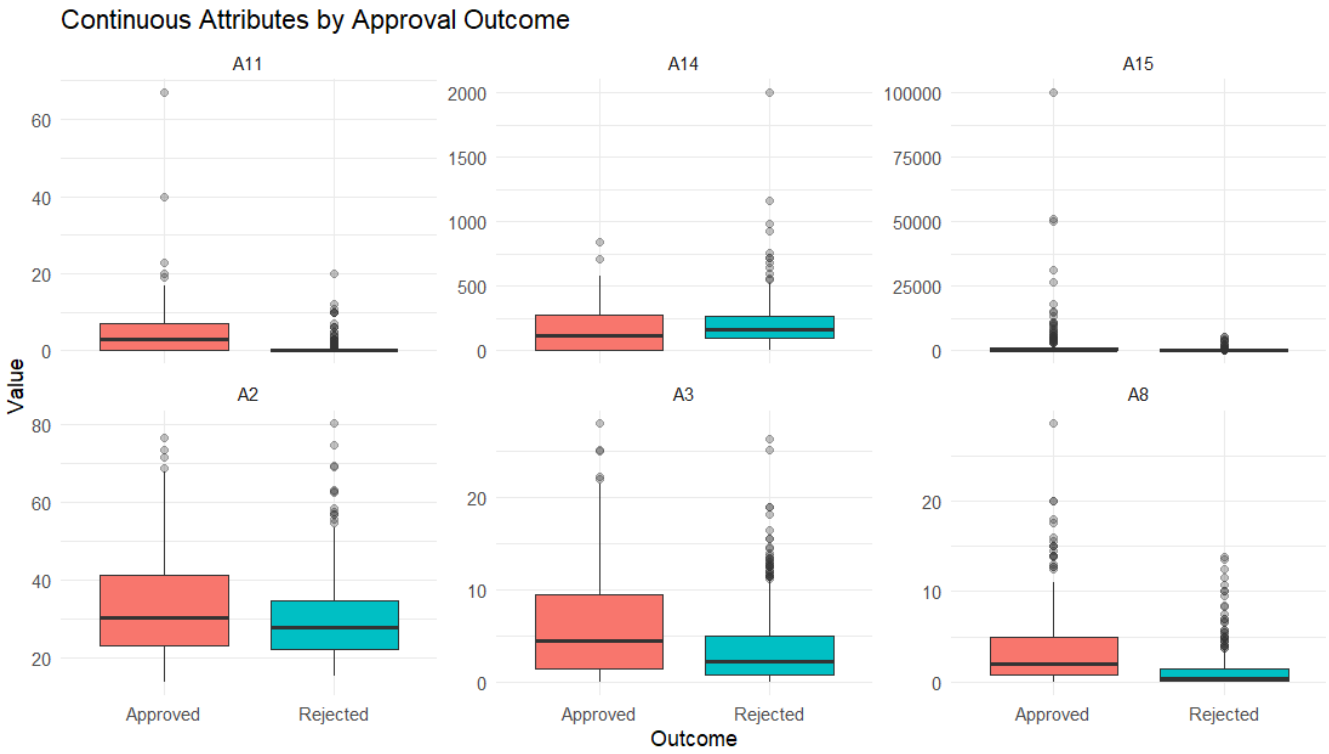
```
> # ============================================================
> # 5+ REQUIRED VISUALIZATIONS
> # ============================================================
>
> # ---- Viz 1: Class balance of the target (A16) ----
> ggplot(credit_clean, aes(x = A16, fill = A16)) +
+   geom_bar() +
+   geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.4) +
+   labs(title = "Distribution of Credit Approval Outcomes",
+        x = "Outcome", y = "Number of Applicants") +
+   theme_minimal() +
+   theme(legend.position = "none")
>
> # ---- Viz 2: Boxplots of continuous attributes by approval outcome ----
> credit_clean %>%
+   select(A16, all_of(num_cols)) %>%
+   pivot_longer(cols = all_of(num_cols), names_to = "attribute", values_to = "value") %>%
+   ggplot(aes(x = A16, y = value, fill = A16)) +
+   geom_boxplot(outlier.alpha = 0.3) +
+   facet_wrap(~ attribute, scales = "free_y") +
+   labs(title = "Continuous Attributes by Approval Outcome",
+        x = "Outcome", y = "Value") +
+   theme_minimal() +
+   theme(legend.position = "none")
>
> # ---- Viz 3: Correlation heatmap of continuous attributes ----
> corr_matrix <- cor(credit_clean[, num_cols], use = "complete.obs")
> corrplot(corr_matrix, method = "color", type = "upper",
+          addCoef.col = "black", tl.col = "black", tl.srt = 45,
+          title = "Correlation Heatmap of Continuous Attributes",
+          mar = c(0, 0, 2, 0))
```

```
> # ---- Viz 4: Approval rate across a key categorical attribute (A9: prior default proxy) ----
> credit_clean %>%
+   group_by(A9, A16) %>%
+   summarise(n = n(), .groups = "drop") %>%
+   group_by(A9) %>%
+   mutate(pct = n / sum(n)) %>%
+   ggplot(aes(x = A9, y = pct, fill = A16)) +
+   geom_col(position = "fill") +
+   scale_y_continuous(labels = scales::percent) +
+   labs(title = "Approval Rate by A9 (Prior Default Proxy)",
+        x = "A9", y = "Proportion", fill = "Outcome") +
+   theme_minimal()
>
> # ---- Viz 5: Missing value pattern ----
> gg_miss_var(credit, show_pct = TRUE) +
+   labs(title = "Missing Values by Attribute")
>
> # ---- (Bonus) Viz 6: Income (A15) distribution, log-scaled, by outcome ----
> ggplot(credit_clean, aes(x = A15 + 1, fill = A16)) +
+   geom_histogram(bins = 40, alpha = 0.7, position = "identity") +
+   scale_x_log10() +
+   labs(title = "Distribution of A15 (Income Proxy, log scale) by Outcome",
+        x = "A15 (log scale)", y = "Count", fill = "Outcome") +
+   theme_minimal()
>
> # ============================================================
> # Save cleaned dataset for DA2 (feature engineering / DB load)
> # ============================================================
> write.csv(credit_clean, "credit_approval_clean.csv", row.names = FALSE)
```

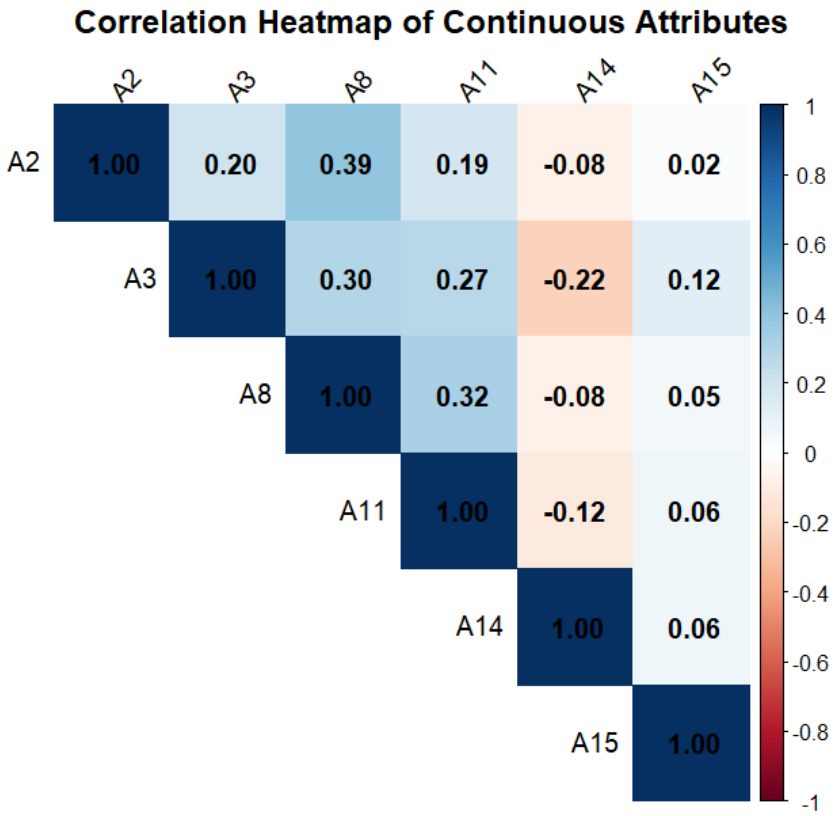
- **Bar chart of approval outcome (A15) counts to check class balance.**



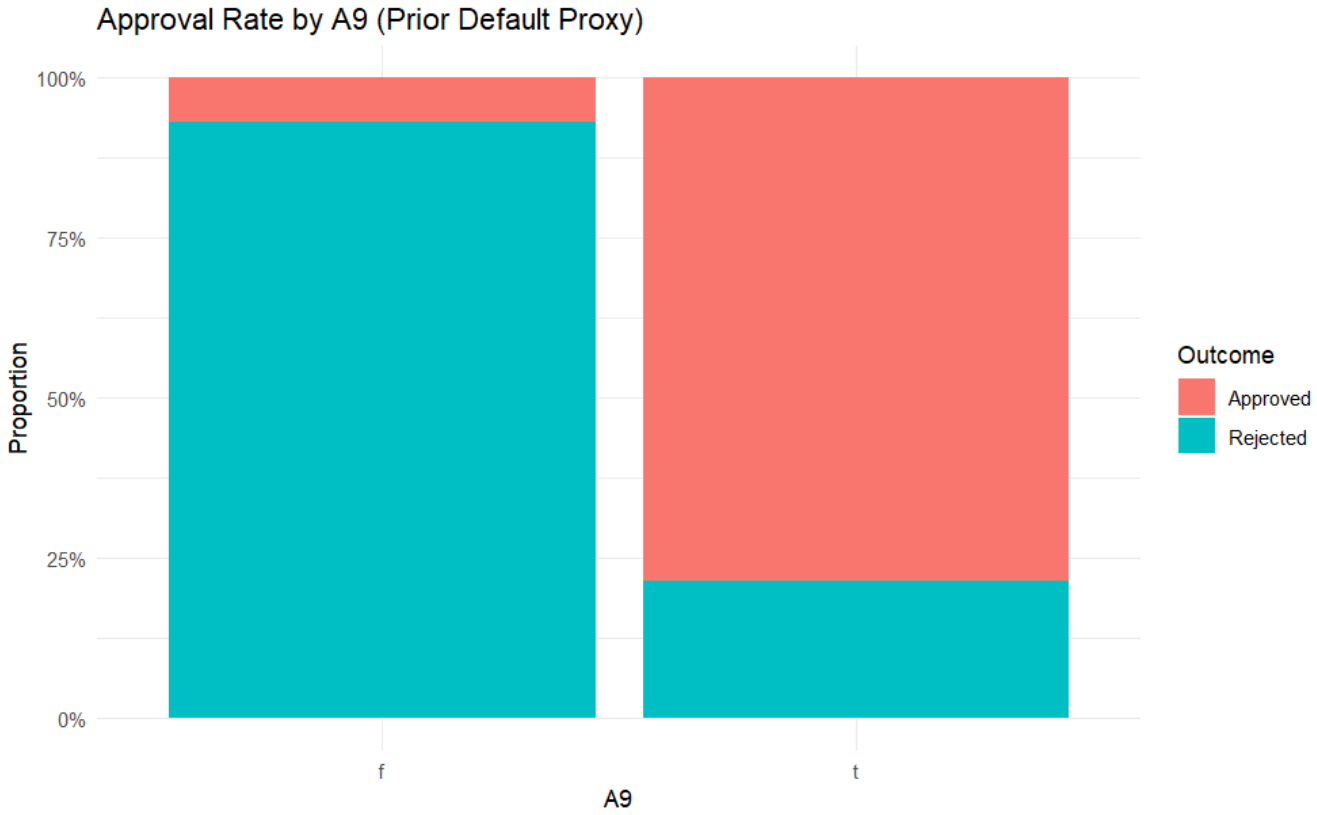
- **Boxplots of key continuous attributes (A2, A3, A8, A11, A15) split by approval outcome.**



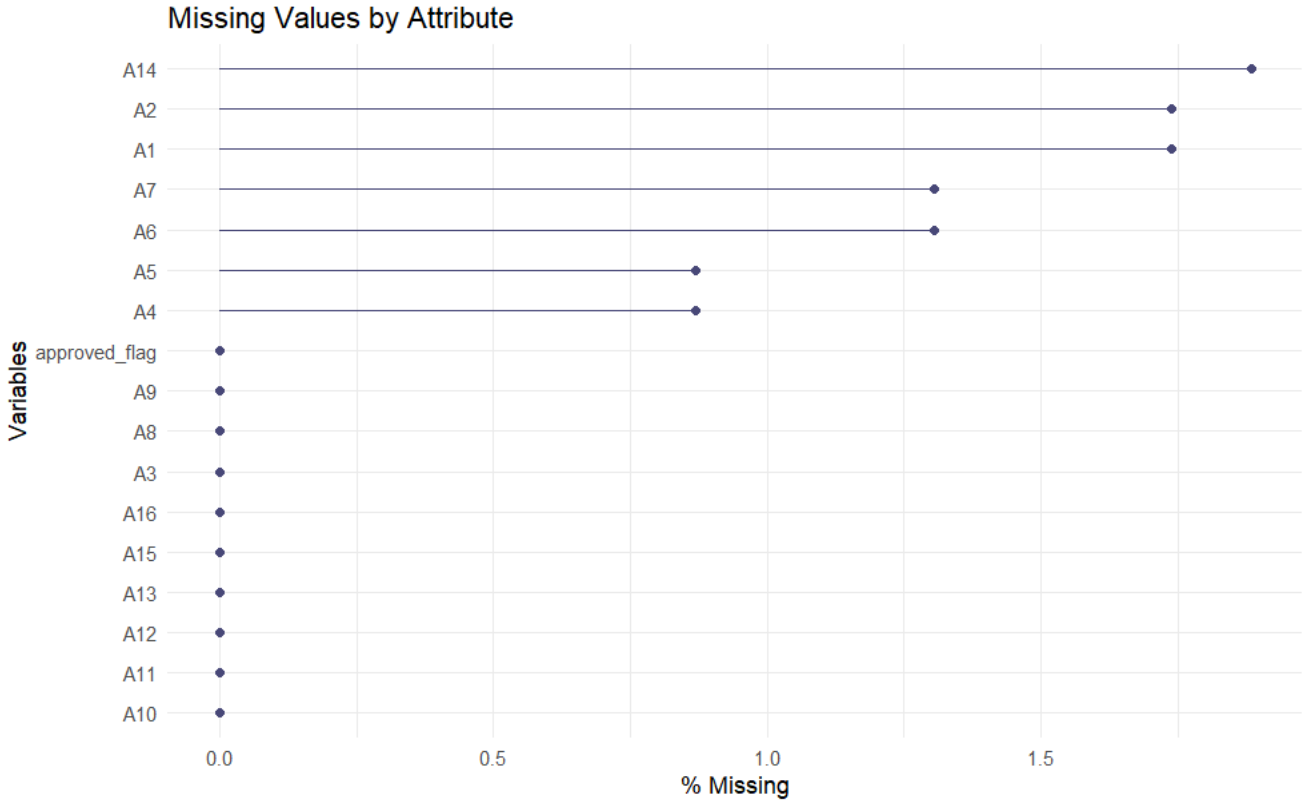
- **Correlation heatmap of continuous attributes.**



- **Stacked bar charts of approval rate across categorical attributes (e.g., A9 prior default, A10 employment status).**



- **Missing value pattern plot to visualize where A1, A2, A7, A14 are missing.**



Data Processing, Database Connectivity & Model Comparison

7. Model Development Overview

This stage moves the credit-approval data into a database, retrieves it with SQL, and turns it into model-ready features, then trains and compares 12 classification models.

The data is the UCI Credit Approval dataset (ID 27): 690 applicants, 15 features (A1 to A15) and a target A16 (approved or rejected). All column names and values are anonymized, so any real-world meaning given to a column in this report is an assumption. The work covers three rubric rows: database connectivity and data retrieval (2 marks), feature engineering and selection (1 mark), and implementation of 10 to 15 ML/DL algorithms (3 marks).

The stages run in this order:

1. Load `crx.data` into two linked database tables.
2. Query the tables with SQL.
3. Pull the joined table back into R.
4. Split it 80/20 into training and test sets.
5. Fill missing values using training-set statistics only.
6. Engineer new features.
7. Select the useful features and save the result for modelling.
8. Train 12 classification models with cross-validation on the training set.
9. Compare the models on the held-out test set.

Four R scripts implement this: `da2_db_config.R` (connection settings), `da2_01_database.R` (stages 1 to 2), `da2_02_features.R` (stages 3 to 7) and `da2_03_models.R` (stages 8 to 9).

8. Database Design

The data lives in two related tables, and the later steps read it from the database rather than from a CSV file. This makes the database a working part of the pipeline, which is what the rubric asks for.

Table	Columns	Role
`applicants`	`id` (primary key), `a1` to `a15`	The 15 features of each applicant
`decisions`	`id` (primary key, references `applicants.id`), `outcome`	Approved or Rejected for each applicant

The `id` is a running number added when the file is loaded. The primary key guarantees each applicant appears once, and the reference guarantees every decision belongs to a real applicant. Missing values (marked `?` in the raw file) are stored as `NULL`.

Choice of database. The default is SQLite, a single file (`credit.db`) that needs no server. PostgreSQL is supported by switching one setting, `USE_POSTGRES`, in `da2_db_config.R`, and entering the connection details there. Column names are lowercase and only standard SQL types are used, so the same queries run on both. MySQL and MongoDB were not used: SQLite and PostgreSQL both fit this small, fixed-shape table, while MongoDB suits flexible, nested records.

9. Loading and SQL Retrieval

R connects to the database with the `DBI` package, loads the file into the two tables, and then retrieves data using seven SQL queries run from R.

Loading has four steps. `crx.data` is read with `na.strings = "?"`, so missing markers become NA instead of a fake category. Each row gets an `id`. The two tables are created with `CREATE TABLE` (dropped first if they exist, so the script can be re-run). The data is then inserted with `dbWriteTable`, and row counts are checked against the expected 690.

Query	What it returns	Why it matters
Q1	First 5 applicants joined with their decision	Confirms the `JOIN` on `id` works
Q2	Count of approved and rejected applicants	Shows the class balance
Q3	Approval rate by `a9`	Uses `GROUP BY` with `CASE WHEN` to compute a percentage
Q4	Approval rate by `a1`	Same grouping, reused later for the fairness audit
Q5	Average `a2` and `a15` by outcome	Compares numeric columns across approved and rejected
Q6	Missing-value count for `a1`, `a2`, `a4`, `a5`, `a6`, `a7`, `a14`	Counts `NULL`s in SQL with `CASE WHEN ... IS NULL`
Q7	Ten applicants with `a15` above 1000, highest first	Inserts the threshold safely with `sqlInterpolate`, which works on both databases

The join behind Q1 to Q5 is the same one used in the next stage to fetch the full modelling table.

Results of the run. The queries returned the following for the 690 applicants.

Query	Result
Row counts	690 applicants and 690 decisions
Q2	307 approved, 383 rejected
Q3	`a9` = t: 78.7% approved (361 applicants). `a9` = f: 7.0% approved (329 applicants)
Q4	`a1` = a: 46.7% approved (210 applicants). `a1` = b: 44.0% approved (468 applicants)
Q5	Approved: average `a2` 33.72, average `a15` 2038.9. Rejected: 29.81 and 198.6
Q6	Missing values: `a1` 12, `a2` 12, `a4` 6, `a5` 6, `a6` 9, `a7` 9, `a14` 13
Q7	The ten largest `a15` values all belong to approved applicants. The largest is 100000

The approval gap by `a9` (71.7 points) is far larger than the gap by `a1` (2.7 points). The Q4 groups leave out the 12 applicants whose `a1` is missing.

10. Train/Test Split and Imputation

The data is split before any cleaning, and missing values are filled using the training set alone. This keeps information from the test set out of the training steps, a mistake known as data leakage.

Split. The 690 applicants are divided 80% training and 20% test with `createDataPartition`. The split is stratified, so both parts keep the same approved-to-rejected ratio, and a fixed seed (42) makes it repeatable. The training set is what the models learn from; the test set is held back as an unseen exam.

In this run the training set holds 553 applicants (246 approved, 307 rejected) and the test set holds 137 (61 approved, 76 rejected). Always predicting “Rejected” would score 55.5% on the test set, which is the baseline every model has to beat.

Imputation. Missing values appear in `A1`, `A2`, `A4`, `A5`, `A6`, `A7` and `A14`, each under 2% of rows. Rows are kept rather than dropped, because the dataset is small.

Data consistency note. The data description in section 4 lists A7 and A14 as the only columns with missing values. The run shows seven columns (A1, A2, A4, A5, A6, A7 and A14), as in the missing-value plot in section 6 and in query Q6, so the seven-column list is the correct one.

- Numeric columns (`A2`, `A14`) receive the training-set median. The median is used because it is not pulled by extreme values.
- Categorical columns receive the training-set most frequent category.
- The same training medians and categories are then applied to the test set.

After imputation, categorical columns are converted to factors with identical levels in both sets, so a category present in only one set cannot break the later steps. A final check confirms no missing values remain in either set.

11. Feature Engineering

Five kinds of new features are built from the 15 original columns, each applied identically to the training and test sets. Because the column meanings are hidden, every feature is justified by the shape of the data (skew, many zeros), not by a real-world interpretation.

New feature	How it is built	Why
`A3_log`, `A8_log`, `A11_log`, `A14_log`, `A15_log`	`log1p(x)`, which is log(1 + x)	Reduces right-skew, where a few very large values dominate. The +1 keeps zeros valid.
`A11_pos`, `A15_pos`	"yes" if the value is above 0, otherwise "no"	Many applicants have 0 in these columns, so having any value at all may matter more than the amount.
`A8_per_A2`	`A8 / A2`	A ratio can capture a relationship that neither column shows alone. `A2` is never 0 (minimum 13.75), so the division is safe.
`A9_and_A10`	"yes" if both `A9` and `A10` are "t"	An interaction flag: two conditions together can matter more than either alone.
`A2_group`	`A2` cut into four bands at the training-set quartiles (Q1 to Q4)	Not used by the models. It is saved as the group column for the fairness audit in the final evaluation.

The interaction is built as a single yes/no flag instead of a four-way combined category, because the combined category would repeat information already in `A9` and `A10` and cause collinearity in models such as logistic regression. The quartile cut-offs for `A2_group` come from the training set and are reused on the test set.

12. Feature Selection

Selection removes features that add noise or repeat information, using three filters in sequence. All are fitted on the training set only.

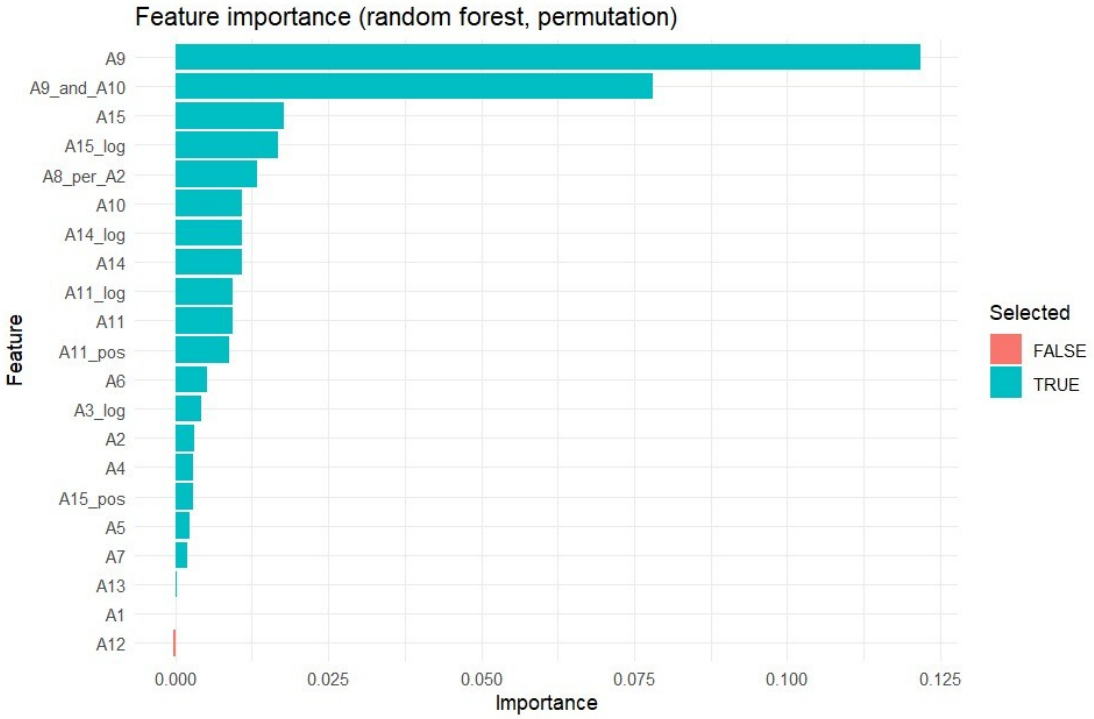
1. **Near-zero variance.** `nearZeroVar` drops columns that are almost constant, because they carry no information to learn from.
2. **High correlation.** Among numeric features, if two have an absolute correlation above 0.90, `findCorrelation` drops one of them. Near-duplicate columns add no information and can destabilise some models.
3. **Permutation importance.** A random forest (`ranger`, 500 trees, seed 42) is trained on the remaining features. Each feature's values are then shuffled in turn, and the drop in prediction accuracy shows how much the model relied on it. A larger drop means a more important feature. Features with positive importance are kept, and at least the top 5 are always kept.

The script saves a bar chart of the importances (`da2_feature_importance.png`) and a table of them (`da2_feature_importance.csv`), and stores the final feature list for the modelling stage.

Result. No column was near-constant. The correlation filter dropped `A8`, `A8_log` and `A3`, leaving 21 candidate features. After ranking, 20 were kept and `A12` was dropped because its importance was negative.

`A9` is by far the strongest predictor (importance 0.122), followed by the `A9_and_A10` flag (0.078). Every other feature scores below 0.02. Three of the top five features are engineered ones: `A9_and_A10`, `A15_log` and `A8_per_A2`. Because `A9_and_A10` is built from `A9` and `A10`, their scores overlap and should not be added together.

The group column planned for the fairness audit, `A1`, stayed among the model features with an importance close to zero.



Permutation importance of each candidate feature; selected features are marked TRUE

13. Model Training

Twelve classification models were trained on the training set and compared on the unseen test set.

The 12 models are logistic regression, elastic-net logistic regression, LDA, KNN, naive Bayes, a decision tree, bagged trees, a random forest, gradient boosting (GBM), a linear SVM, an RBF SVM and a neural network. The neural network is a single-hidden-layer perceptron, not a deep network.

- **Cross-validation.** Each model is scored with 5-fold cross-validation repeated twice on the training set. All models use the same folds, so their scores are directly comparable.
- **Preparation.** Before fitting, categorical features become 0/1 dummy columns, columns with no variation are removed, and numeric columns are centred and scaled.
- **Tuning.** Each model's settings (such as tree depth or the number of neighbours) are chosen by cross-validated AUC.
- **Random forest grid.** The `mtry` values are set explicitly below the column count, because the zero-variance step can remove columns inside a fold, and the default grid then failed on some folds.

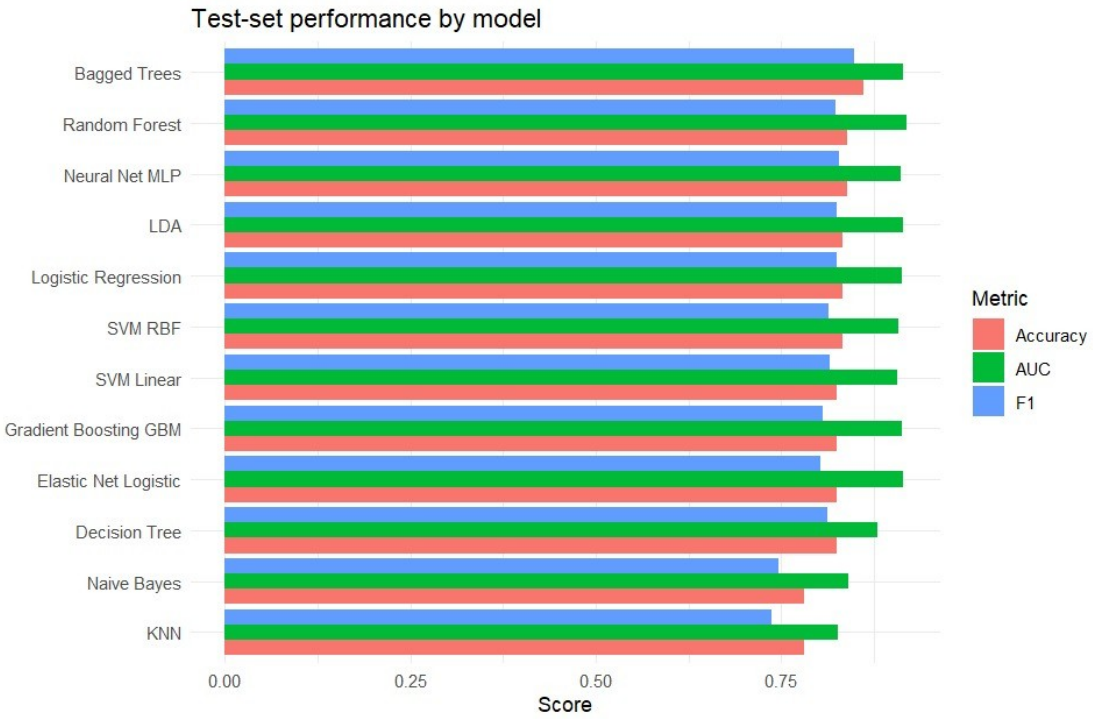
- **Test metrics.** Accuracy, precision, recall and F1 treat Approved as the positive class. AUC measures how well a model ranks approved applicants above rejected ones, where 0.5 is a coin flip and 1.0 is perfect.

14. Results

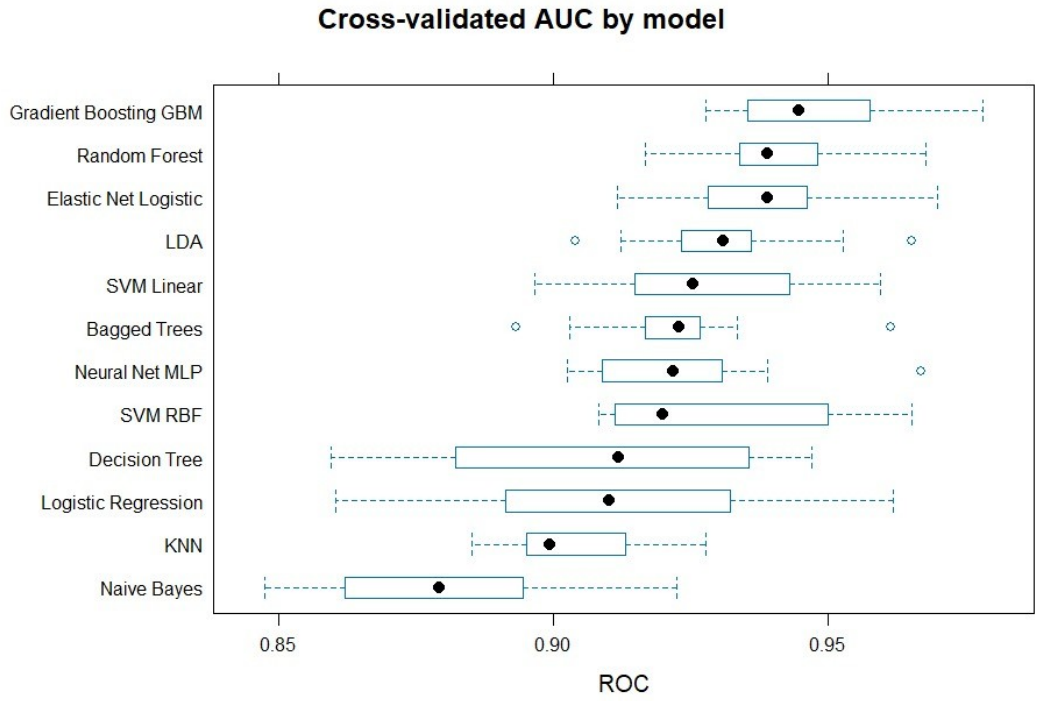
Seven models fall within 0.007 AUC of each other on the test set, so the data does not clearly favour one model. Random Forest is highest at 0.918. Test metrics use the 137 held-out applicants, sorted by test AUC; CV AUC is the cross-validated AUC on the training set.

Model	Accuracy	Precision	Recall	F1	Test AUC	CV AUC
Random Forest	0.839	0.810	0.836	0.823	0.918	0.939
Elastic Net Logistic	0.825	0.803	0.803	0.803	0.914	0.938
LDA	0.832	0.771	0.885	0.824	0.914	0.932
Bagged Trees	0.861	0.828	0.869	0.848	0.914	0.922
Logistic Regression	0.832	0.771	0.885	0.824	0.913	0.912
Gradient Boosting GBM	0.825	0.794	0.820	0.806	0.912	0.949
Neural Net MLP	0.839	0.791	0.869	0.828	0.911	0.924
SVM RBF	0.832	0.806	0.820	0.813	0.908	0.929
SVM Linear	0.825	0.768	0.869	0.815	0.906	0.928
Decision Tree	0.825	0.776	0.852	0.812	0.880	0.907
Naive Bayes	0.781	0.772	0.721	0.746	0.840	0.880
KNN	0.781	0.792	0.689	0.737	0.826	0.903

The first plot shows the test accuracy, F1 and AUC for each model. The second shows how each model’s AUC varied across the 10 cross-validation folds.



Test-set accuracy, F1 and AUC by model



Cross-validated AUC by model

- **Baseline.** Always predicting "Rejected" scores 55.5% accuracy on the test set. All 12 models beat it; the two lowest, KNN and naive Bayes, reach 78.1%.
- **Best accuracy.** Bagged Trees has the highest accuracy (0.861) and F1 (0.848), though its AUC (0.914) is level with LDA and elastic net.
- **Simple models keep up.** Logistic regression reaches an AUC of 0.913, against 0.918 for the random forest. This fits with the earlier finding that one feature, A9, carries most of the signal.
- **Small test set.** One applicant moves accuracy by about 0.7 points. The ten models between 0.825 and 0.861 differ by five applicants in total, so those gaps are small enough to be noise.

- **Cross-validation vs test.** CV AUC is above test AUC for every model except logistic regression, which is level (0.912 and 0.913). The largest drop is KNN, from 0.903 to 0.826.

These differences are examined statistically in the fairness analysis.

15. Outputs, Limitations and Next Stage

The modelling stage saves the following files for the fairness audit and final report.

- `credit.db` with the tables `applicants`, `decisions` and `model_results` (or the PostgreSQL equivalent).
- `da2_data.rds`, holding the imputed and feature-engineered training and test sets plus the selected feature list.
- `da2_feature_importance.csv` and `da2_feature_importance.png`, showing how each feature ranked.
- `da2_model_results.csv` and `da2_model_comparison.png`, with the test metrics for each model.
- `da2_test_predictions.csv`, with every model's predicted class and probability for each test applicant, plus the `A1` and `A2_group` columns.

Limitations and assumptions

- The column meanings are unknown, so engineered features rest on the data's shape, and any group built from `A1` or `A2` is an anonymous group, not a confirmed gender or age group.
- `A16` is the bank's past decision, not a verified record of later default, so any bias in those decisions is built into the labels.
- The test set holds 137 applicants, so one applicant moves accuracy by about 0.7 points, and
-
- The model comparison has not been tested statistically, and no fairness metrics have been computed yet; these are covered in the final evaluation.

Final Evaluation, Fairness Audit & Project Demonstration

16. Final Evaluation and Method

The final evaluation uses the 12 trained models and their predictions on the 137 held-out test applicants, and asks two questions: are the differences in model performance real, and do the models treat applicant groups equally?

Main findings

- Ten of the 12 models reach 82.5% to 86.1% accuracy, against a 55.5% baseline. The Random Forest has the highest test AUC (0.918), but it is statistically tied with nine of the other models. Only KNN and Naive Bayes are significantly worse.
- No model shows a statistically significant fairness gap once the p-values are corrected for the 24 model-group tests per metric (smallest adjusted p-value 0.156 for the approval-rate gap, 0.492 for the equal opportunity gap).
- Approval-rate gaps by A2 quartile are about the size of the gap already present in the true labels, and equal opportunity gaps by A2 are within chance for every model.
- Bagged Trees, Random Forest and the Decision Tree have the largest equal opportunity gaps by A1 (0.167 to 0.211). This is a lead for further work, not a confirmed bias.
- Removing A1 changes at most 1.5% of predictions and does not remove the gaps.

Groups audited. Two group columns are used, and both are anonymous. The dataset hides what each column means, so they are not confirmed gender or age groups.

- A1: two groups, a (45 test applicants) and b (92).
- A2 quartile: four groups, Q1 to Q4 (35, 33, 39 and 30 test applicants), cut at the training-set quartiles of A2.

Fairness metrics. "Approved" is the positive class.

Metric	Definition	Question it answers
Approval-rate gap (demographic parity)	Largest difference between groups in the share of applicants the model approves	Are the groups approved at similar rates?
Equal opportunity gap	Largest difference between groups in the true-positive rate, the share of truly approved applicants the model approves	Among applicants who really should be approved, are the groups treated equally?

The scripts also record the equalized-odds gap and the disparate-impact ratio (lowest approval rate divided by highest). Two metrics are used because an approval-rate gap alone can be legitimate: if one group truly has more approved applicants, a fair model should approve it more often. Equal opportunity compares only applicants with the same true outcome, so it separates such legitimate differences from unequal treatment. Equal opportunity and equalized odds belong to the separation family of criteria discussed by Kozodoi et al. (section 3, reference 2).

Statistical treatment. With only 137 test applicants, every gap is noisy, so each one is reported with its uncertainty.

- **Confidence interval.** A 95% bootstrap interval from 1,000 resamples of the test applicants. A gap cannot be negative, so these intervals are skewed upward.
- **Chance test.** The group labels are shuffled 2,000 times to see what gap appears when the group has no real link to the predictions. The p-value is the share of shuffles with a gap at least as large as the observed one. The "chance" value in the tables is the average gap from these shuffles.
- **Multiple tests.** There are 12 models and 2 group columns, so 24 tests per metric. P-values are corrected with the Holm method ("adj. p").

- **Model comparison.** The model with the highest test AUC is the reference. A paired DeLong test compares AUCs, and an exact McNemar test compares which applicants each model classifies correctly, both on the same 137 applicants.

17. Final Performance Comparison and Statistical Analysis

Test-set results for all 12 models, sorted by AUC. The intervals are 95% bootstrap intervals. CV AUC is the cross-validated AUC on the training set.

Model	Accuracy (95% interval)	Test AUC (95% interval)	CV AUC
Random Forest	0.839 (0.774 to 0.898)	0.918 (0.864 to 0.961)	0.939
Elastic Net Logistic	0.825 (0.759 to 0.883)	0.914 (0.864 to 0.956)	0.938
LDA	0.832 (0.766 to 0.891)	0.914 (0.866 to 0.956)	0.932
Bagged Trees	0.861 (0.796 to 0.912)	0.914 (0.862 to 0.960)	0.922
Logistic Regression	0.832 (0.766 to 0.891)	0.913 (0.863 to 0.952)	0.912
Gradient Boosting GBM	0.825 (0.759 to 0.891)	0.912 (0.862 to 0.954)	0.949
Neural Net MLP	0.839 (0.774 to 0.898)	0.911 (0.858 to 0.954)	0.924
SVM RBF	0.832 (0.766 to 0.891)	0.908 (0.852 to 0.952)	0.929
SVM Linear	0.825 (0.759 to 0.883)	0.906 (0.855 to 0.952)	0.928
Decision Tree	0.825 (0.766 to 0.883)	0.880 (0.821 to 0.935)	0.907
Naive Bayes	0.781 (0.708 to 0.847)	0.840 (0.770 to 0.900)	0.880
KNN	0.781 (0.715 to 0.847)	0.826 (0.753 to 0.890)	0.903

The Random Forest has the highest test AUC, so it is the reference for the paired tests. “Ref. only right” counts the test applicants the Random Forest classifies correctly and the other model does not, and “Model only right” is the reverse.

Model	AUC	AUC diff.	DeLong adj. p	Ref. only right	Model only right	McNemar adj. p
KNN	0.826	0.092	0.005	11	3	0.631
Naive Bayes	0.840	0.077	0.043	13	5	0.963
Decision Tree	0.880	0.038	0.492	5	3	1.000
SVM RBF	0.908	0.009	1.000	3	2	1.000
SVM Linear	0.906	0.012	1.000	6	4	1.000
Gradient Boosting GBM	0.912	0.006	1.000	4	2	1.000
Neural Net MLP	0.911	0.006	1.000	4	4	1.000
Bagged Trees	0.914	0.004	1.000	2	5	1.000
Elastic Net Logistic	0.914	0.003	1.000	3	1	1.000
LDA	0.914	0.004	1.000	6	5	1.000
Logistic Regression	0.913	0.005	1.000	5	4	1.000

- The AUC of KNN and Naive Bayes is significantly lower than the Random Forest’s after correction (adjusted p = 0.005 and 0.043). Their accuracy is not significantly different (McNemar adjusted p = 0.631 and 0.963).
- For the other nine models, the data cannot tell them apart from the Random Forest. All adjusted p-values are 0.492 or higher, and the AUC differences are 0.038 or less.
- Every model beats the 55.5% majority-class baseline. The two weakest, KNN and Naive Bayes, reach 78.1%.
- Cross-validated AUC is above test AUC for every model except logistic regression, which is level (0.912 and 0.913). The largest drop is KNN, from 0.903 to 0.826. With 137 test applicants, part of this gap is sampling noise.

- Model choice among the top ten is therefore a judgment call. Logistic regression (AUC 0.913) is nearly as good as the Random Forest and is easy to explain.

18. Fairness Audit Results

Base rates. Before looking at the models, here is how the true labels differ between groups in the test set.

Group column	Group	Test applicants	True approval rate
A1	a	45	0.511
A1	b	92	0.413
A2 quartile	Q1	35	0.343
A2 quartile	Q2	33	0.424
A2 quartile	Q3	39	0.410
A2 quartile	Q4	30	0.633

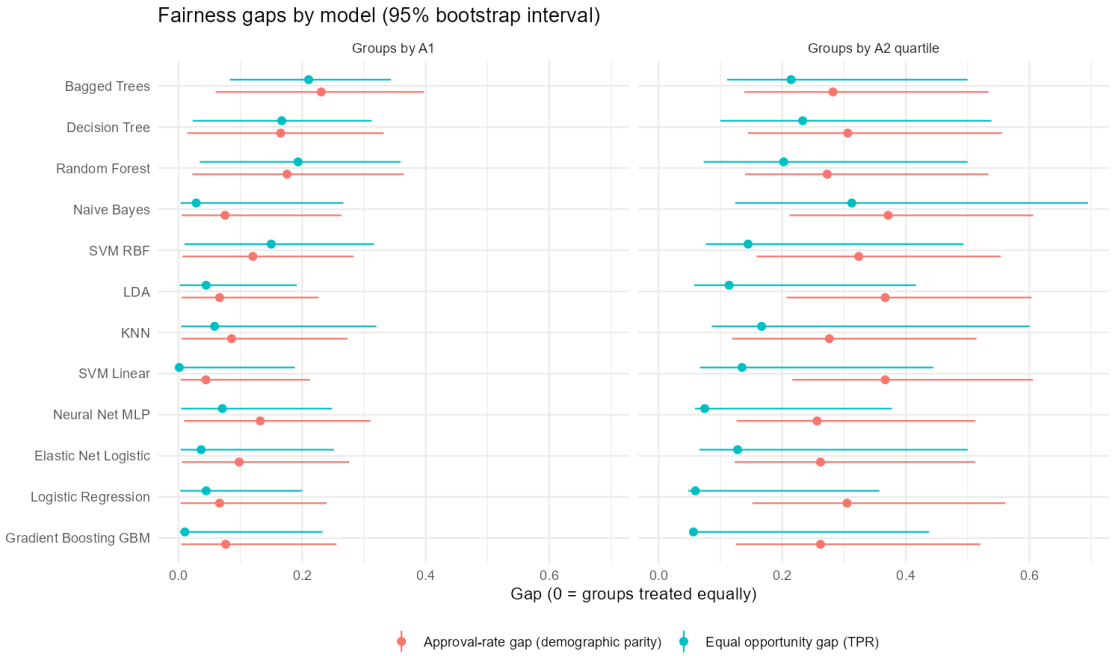
The true approval rate is already 9.8 points higher for A1 group a than for group b, and 29.0 points higher for Q4 than for Q1. Some gap in model decisions is therefore expected, which is why equal opportunity is read alongside the approval-rate gap.

Fairness gaps by A1. DP = approval-rate gap, EO = equal opportunity gap. “Chance” is the gap expected from shuffled group labels. Raw p-values are shown; none of the adjusted p-values is below 0.05.

Model	DP gap	DP chance	DP p	EO gap	EO chance	EO p
Logistic Regression	0.066	0.073	0.473	0.045	0.070	0.600
Elastic Net Logistic	0.098	0.075	0.380	0.037	0.089	0.746
LDA	0.066	0.071	0.482	0.045	0.070	0.604
KNN	0.086	0.071	0.354	0.058	0.105	0.673
Naive Bayes	0.075	0.070	0.444	0.029	0.099	0.831
Decision Tree	0.165	0.072	0.104	0.167	0.081	0.097
Bagged Trees	0.231	0.074	0.015	0.211	0.076	0.020
Random Forest	0.176	0.071	0.050	0.193	0.082	0.056
Gradient Boosting GBM	0.076	0.075	0.482	0.010	0.085	0.929
SVM Linear	0.044	0.075	0.732	0.001	0.075	1.000
SVM RBF	0.120	0.071	0.193	0.150	0.086	0.174
Neural Net MLP	0.132	0.075	0.219	0.071	0.076	0.450

Fairness gaps by A2 quartile.

Model	DP gap	DP chance	DP p	EO gap	EO chance	EO p
Logistic Regression	0.305	0.176	0.048	0.060	0.173	0.976
Elastic Net Logistic	0.262	0.175	0.140	0.128	0.219	0.828
LDA	0.367	0.174	0.010	0.114	0.173	0.754
KNN	0.276	0.170	0.084	0.167	0.251	0.769
Naive Bayes	0.371	0.173	0.006	0.312	0.244	0.249
Decision Tree	0.306	0.181	0.065	0.233	0.195	0.305
Bagged Trees	0.282	0.176	0.086	0.214	0.184	0.349
Random Forest	0.273	0.177	0.109	0.202	0.202	0.462
Gradient Boosting GBM	0.262	0.178	0.138	0.056	0.212	0.984
SVM Linear	0.367	0.179	0.010	0.135	0.181	0.697
SVM RBF	0.324	0.178	0.045	0.145	0.210	0.768
Neural Net MLP	0.256	0.177	0.154	0.075	0.183	0.954



Fairness gaps by model with 95% bootstrap intervals

Reading the tables.

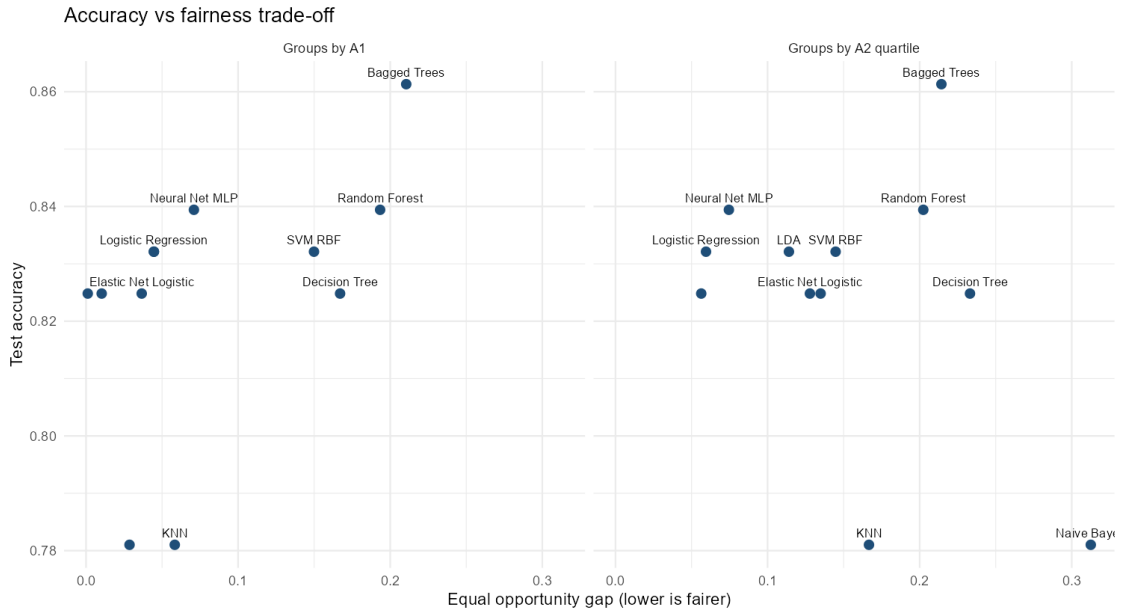
- **Approval-rate gaps.** Six of the 24 tests are below 0.05 before correction: five by A2 quartile (logistic regression, LDA, Naive Bayes, linear SVM, RBF SVM) and Bagged Trees by A1. After Holm correction none is significant (smallest adjusted p = 0.156).
- **A2 quartile.** The model approval-rate gaps (0.256 to 0.371) are about the size of the 0.290 gap already in the true labels. Equal opportunity gaps by A2 are within chance for every model (all p of 0.249 or higher). So the A2 differences are consistent with the data, not with the models introducing a new gap.
- **A1.** Bagged Trees has the only equal opportunity p-value below 0.05 (0.020, adjusted 0.492). The Random Forest is borderline (0.056). For both, the model gap (0.231 and 0.176 in approval rate) is larger than the true-label gap of 0.098.

Worked example: Random Forest. The per-group numbers behind its A1 and A2 gaps. N is the number of test applicants in the group, TPR is the share of truly approved applicants approved by the model, and FPR is the share of truly rejected applicants approved.

Group column	Group	N	Truly approved	Approval rate	TPR	FPR	Accuracy
A1	a	45	23	0.58	0.96	0.18	0.89
A1	b	92	38	0.40	0.76	0.15	0.82
A2 quartile	Q1	35	12	0.40	0.92	0.13	0.89
A2 quartile	Q2	33	14	0.39	0.71	0.16	0.79
A2 quartile	Q3	39	16	0.41	0.81	0.13	0.85
A2 quartile	Q4	30	19	0.67	0.89	0.27	0.83

For A1, the Random Forest approves 22 of the 23 truly approved applicants in group a (0.96) but 29 of 38 in group b (0.76). Group a is also approved at 0.58, above its true rate of 0.51, while group b is approved at 0.40, close to its true rate of 0.41. These groups contain only 23 and 38 truly approved applicants, so the gap is imprecise (95% interval 0.034 to 0.359) and not significant after correction.

19. Accuracy vs Fairness Trade-off



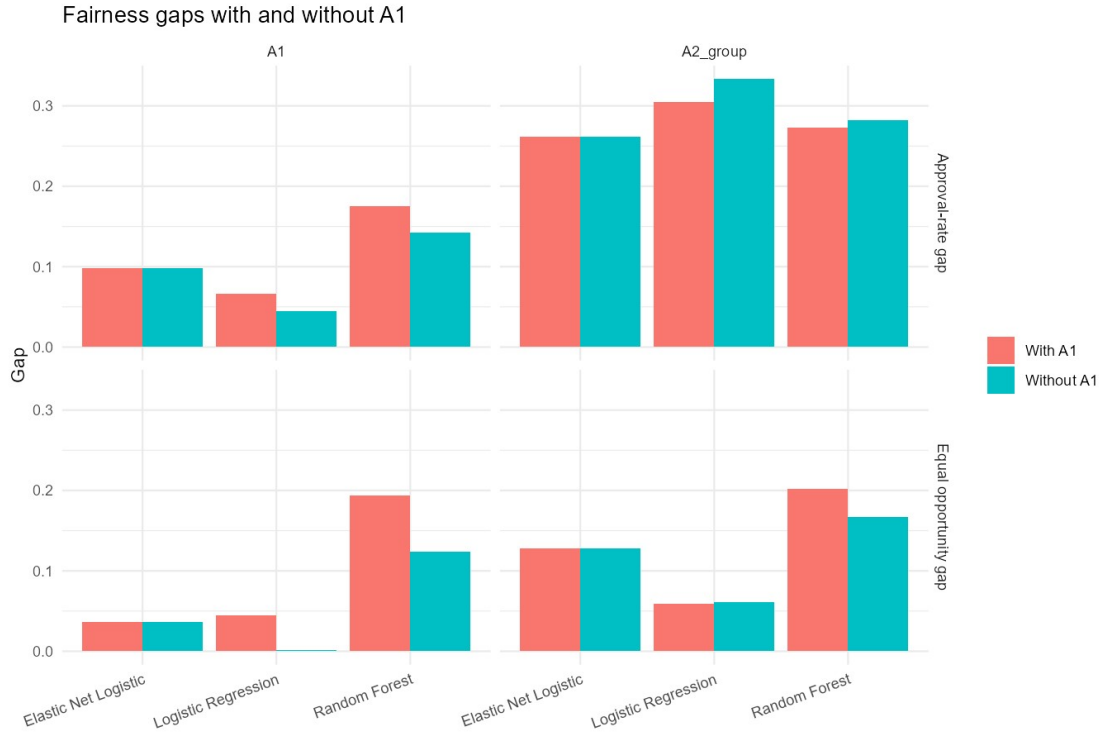
Test accuracy against equal opportunity gap, one point per model

- The most accurate model, Bagged Trees (0.861), also has the largest A1 equal opportunity gap (0.211).
- Linear SVM, GBM and Elastic Net score 0.825 with A1 gaps of 0.001, 0.010 and 0.037. That is five applicants fewer correct than Bagged Trees (118 against 113 out of 137).
- By A2 quartile there is no clear pattern. The two most accurate models, Bagged Trees and Random Forest, have gaps of 0.214 and 0.202, but the Neural Net MLP matches the Random Forest's accuracy (0.839) with a gap of 0.075. Naive Bayes has both low accuracy (0.781) and the largest gap (0.312).
- This suggests a mild trade-off on A1, but the intervals overlap heavily (Bagged Trees 0.083 to 0.344, linear SVM 0.002 to 0.188), so it cannot be treated as established.

20. Does Removing A1 Make the Models Fairer?

A common first idea is to delete the group column. To test it, three models (logistic regression, elastic net and Random Forest) were retrained with and without A1, on the same cross-validation folds. "Acc." is accuracy and "Changed" is the share of test predictions that differ between the two variants.

Model	Variant	Acc.	AUC	A1 DP gap	A1 EO gap	A2 DP gap	A2 EO gap	Changed
Logistic Regression	With A1	0.832	0.913	0.066	0.045	0.305	0.060	0.7%
Logistic Regression	Without A1	0.825	0.913	0.044	0.001	0.333	0.061	0.7%
Elastic Net Logistic	With A1	0.825	0.914	0.098	0.037	0.262	0.128	0.0%
Elastic Net Logistic	Without A1	0.825	0.914	0.098	0.037	0.262	0.128	0.0%
Random Forest	With A1	0.839	0.918	0.176	0.193	0.273	0.202	1.5%
Random Forest	Without A1	0.839	0.919	0.143	0.124	0.282	0.167	1.5%



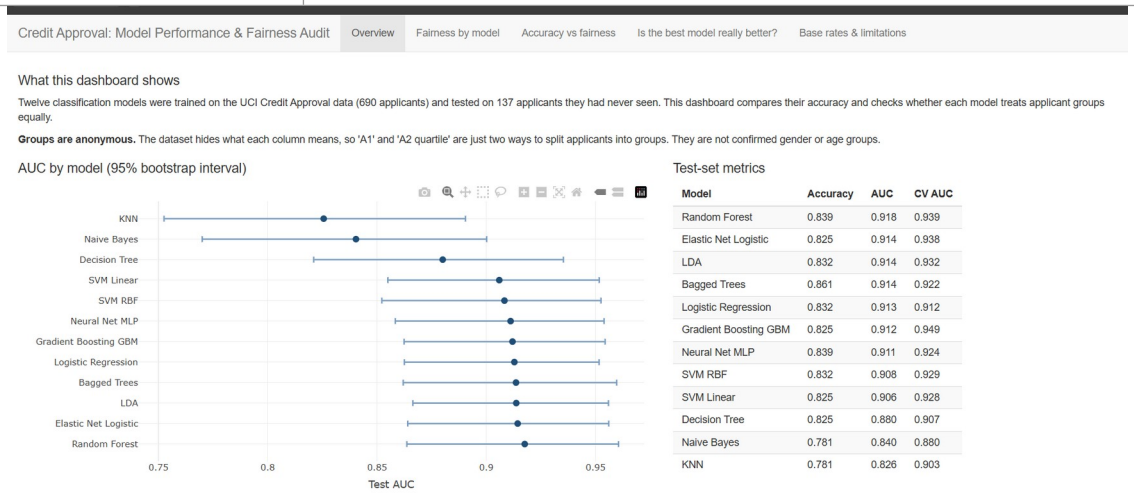
Fairness gaps with and without A1

- Removing A1 changed 0 to 2 of the 137 test predictions (0% to 1.5%), and accuracy moved by at most 0.007. This fits with A having near-zero importance in modeling.
- The Random Forest's A1 gaps fell (equal opportunity 0.193 to 0.124) but stayed sizeable, so the gap is carried by other features. That is the usual weakness of simply dropping a sensitive column.
- The same change of two predictions moved the gap by about 7 points, which shows how unstable gaps are with 137 test applicants.

21. Interactive Dashboard

The dashboard is an R Shiny and Plotly app (`da3_dashboard/app.R`). It reads only the saved result tables, so it opens in seconds and nothing is retrained. Run it with `shiny::runApp("da3_dashboard")`.

Tab	What it shows
Overview	AUC of every model with its 95% interval, and the test-set metrics table
Fairness by model	Choose a model and a group column to see both gaps with intervals, the gap expected by chance, p-values, and per-group approval and true-positive rates
Accuracy vs fairness	One point per model; choose the gap type, group column and performance measure; hover for the model name
Is the best model really better?	The DeLong and McNemar comparisons against the reference model
Base rates & limitations	True approval rates by group and the main limitations



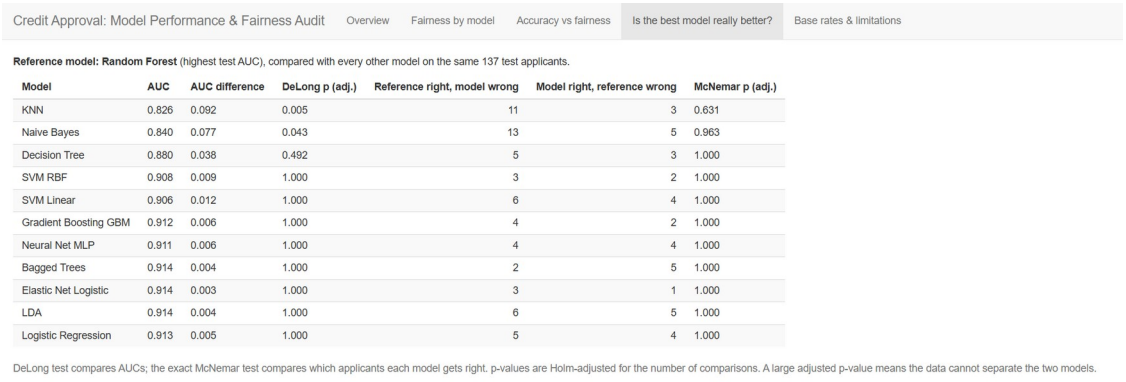
Overview tab



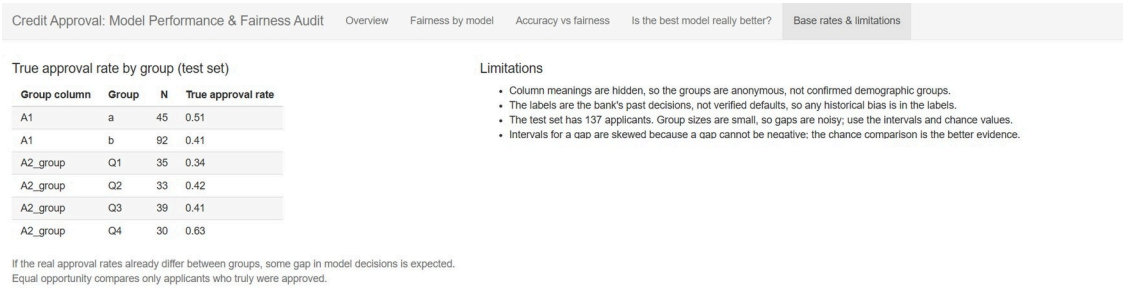
Fairness by model tab: Random Forest, groups by A1



Fairness by model tab: Random Forest, groups by A2 quartile



Model comparison tests tab



Base rates and limitations tab

22. Result Interpretation and Discussion

Performance. Twelve models were compared on the same 137 held-out applicants. Random Forest had the highest observed test AUC (0.918), but several models were close: Elastic Net Logistic and LDA each reached 0.914, Logistic Regression 0.913, Gradient Boosting 0.912, and Neural Net MLP 0.911. This narrow range means the highest AUC alone is not enough to claim Random Forest is best for every purpose.

Statistical comparison. The paired DeLong test found a corrected difference in AUC between Random Forest and KNN (adjusted p = 0.005) and between Random Forest and Naive Bayes (adjusted p = 0.043). The other AUC comparisons were not significant after correction. Adjusted McNemar comparisons did not establish a significant difference in which applicants the models classified correctly. Random Forest leads on observed AUC, but it is not proven superior to most strong alternatives.

Fairness. No tested gap remained statistically significant after correction for multiple comparisons. This is not proof that the models are fair. The test set contains only 137 applicants, and group sizes range from 30 to 92, so estimates are uncertain and moderate differences may be difficult to detect.

Fairness findings by group. For A2_group, the historical approval rate was about 34% in Q1, 42% in Q2, 41% in Q3 and 63% in Q4. Model approval rates largely reflect this pattern already present in the historical labels. Equal opportunity compares only applicants whose recorded label was Approved; the audit did not find a corrected statistically significant difference for this grouping.

For A1, the largest observed equal-opportunity gaps were in Bagged Trees (0.211), Random Forest (0.193), Decision Tree (0.167) and RBF SVM (0.150). These are leads for a larger follow-up study, not proof of discrimination: the corrected tests did not establish significance. Removing A1 also did not eliminate all group differences, because other inputs can carry related information.

23. Conclusion, Limitations and Future Scope

Conclusion. The project prepared and stored the credit-approval data, engineered model inputs, compared 12 classifiers, and built a fairness audit with an interactive dashboard. Random Forest produced the highest observed test AUC (0.918), but paired tests did not show it to be statistically better than most other strong models. The fairness audit found no gap that remained significant after multiple-testing correction. The correct interpretation is that this test set did not provide strong statistical evidence of a group gap; it does not establish that the models are fair.

Limitations

- The meanings of A1 and A2 are anonymized, so the groups cannot be treated as confirmed demographic groups.
- The target is the bank's historical decision, not verified repayment or default. Historical bias may therefore be present in the labels.
- The test set contains 137 applicants, with small group sizes and wide uncertainty intervals.
- Many model and fairness comparisons are made. Multiple-testing correction reduces false positives, but limited sample size also reduces power to detect moderate effects.
- Results describe this dataset and split; they may not generalize to other applicants or lending settings.

Future scope

- Evaluate out-of-fold predictions across more of the 690 applications to increase the evidence available for each group.
- Test mitigation approaches such as reweighting or threshold adjustment and compare both performance and fairness.
- Use data with documented group attributes and verified repayment outcomes.
- Examine individual explanations and assess whether they differ across groups.
- Validate the system on additional samples before considering any real-world lending use.

24. Project Structure and Code Organization

Every script reads the saved output of the previous stage, so a stage only needs to be re-run if its code or data changes.

Stage	Script	Main outputs
Preprocessing	`da1_preprocessing.R`, `da1_eda.R`	`credit_approval_clean.csv`, six EDA plots
Database	`da2_db_config.R`, `da2_01_database.R`	`credit.db` (tables `applicants`, `decisions`)
Feature engineering	`da2_02_features.R`	`da2_data.rds`, `da2_feature_importance.csv` and `.png`
Model training	`da2_03_models.R`	`da2_model_results.csv`, `da2_test_predictions.csv`, `da2_model_comparison.png`, table `model_results`
Fairness audit	`da3_helpers.R`, `da3_01_fairness.R`	`da3_model_summary.csv`, `da3_group_metrics.csv`, `da3_fairness_summary.csv`, `da3_model_tests.csv`, `da3_base_rates.csv`, `da3_fairness_gaps.png`, `da3_tradeoff.png`
experiment	`da3_02_without_A1.R`	`da3_unawareness.csv`, `da3_unawareness.png`
dashboard	`da3_dashboard/app.R`	Interactive Shiny dashboard (reads `da3_dashboard/data`)

Database/model code is in Appendix A; fairness-audit and dashboard code is in Appendix B.

Appendix A · Database & Model Training Code

The four scripts below are run in this order: `da2_01_database.R`, `da2_02_features.R`, `da2_03_models.R`. `da2_db_config.R` is loaded by all three.

da2_db_config.R

```
# ============================================================
# modeling - Shared database connection settings
# All modeling scripts call source("da2_db_config.R") and then get_con()
# ============================================================

library(DBI)

# FALSE = SQLite (a single file "credit.db", zero setup)
# TRUE  = PostgreSQL (needs a running server + the RPostgres package)
USE_POSTGRES <- FALSE

get_con <- function() {
  if (USE_POSTGRES) {
    dbConnect(
      RPostgres::Postgres(),
      dbname   = "credit_db",     # create this database first in pgAdmin / psql
      host     = "localhost",
      port     = 5432,
      user     = "postgres",
      password = "YOUR_PASSWORD"  # <- change this
    )
  } else {
    dbConnect(RSQLite::SQLite(), "credit.db")
  }
}
```

da2_01_database.R

```
# ============================================================
# modeling - Part 1: Database connectivity & data retrieval
# Loads crx.data into two related tables, then queries them with SQL
# ============================================================

source("da2_db_config.R")
```

```
# ---- 1. Read the raw file ----
# Lowercase column names (a1..a16) so the same SQL works in SQLite AND PostgreSQL
raw <- read.csv(
  "crx.data",
  header = FALSE,
  na.strings = "?",           # "?" becomes NA, which is stored as NULL in the database
  col.names = paste0("a", 1:16),
  stringsAsFactors = FALSE
)
raw$id <- seq_len(nrow(raw))  # give every applicant a unique id (primary key)


# ---- 2. Connect ----
con <- get_con()


# ---- 3. Create the schema (two tables linked by id) ----
dbExecute(con, "DROP TABLE IF EXISTS decisions")
dbExecute(con, "DROP TABLE IF EXISTS applicants")


dbExecute(con, "
CREATE TABLE applicants (
  id  INTEGER PRIMARY KEY,
  a1  TEXT,
  a2  DOUBLE PRECISION,
  a3  DOUBLE PRECISION,
  a4  TEXT,
  a5  TEXT,
  a6  TEXT,
  a7  TEXT,
  a8  DOUBLE PRECISION,
  a9  TEXT,
  a10 TEXT,
  a11 INTEGER,
  a12 TEXT,
  a13 TEXT,
  a14 INTEGER,
  a15 INTEGER
)")


dbExecute(con, "
CREATE TABLE decisions (
  id      INTEGER PRIMARY KEY REFERENCES applicants(id),
  outcome TEXT
)")


# ---- 4. Insert the data ----
applicants_df <- raw[, c("id", paste0("a", 1:15))]
decisions_df  <- data.frame(
  id      = raw$id,
  outcome = ifelse(raw$a16 == "+", "Approved", "Rejected")
)


dbWriteTable(con, "applicants", applicants_df, append = TRUE)
dbWriteTable(con, "decisions",  decisions_df,  append = TRUE)


# ---- 5. Sanity checks ----
dbListTables(con)
dbGetQuery(con, "SELECT COUNT(*) AS n_applicants FROM applicants")
dbGetQuery(con, "SELECT COUNT(*) AS n_decisions  FROM decisions")


# ============================================================
# DATA RETRIEVAL - SQL queries run from R
# ============================================================


# Q1: first 5 applicants joined with their decision
dbGetQuery(con, "
  SELECT a.id, a.a1, a.a2, a.a9, a.a10, a.a15, d.outcome
  FROM applicants a
  JOIN decisions d ON a.id = d.id
  ORDER BY a.id
  LIMIT 5")


# Q2: overall approval / rejection counts
dbGetQuery(con, "
  SELECT outcome, COUNT(*) AS n
  FROM decisions
  GROUP BY outcome")


# Q3: approval rate by a9 (a two-value categorical column)
dbGetQuery(con, "
  SELECT a.a9,
         COUNT(*) AS n,
         ROUND(100.0 * SUM(CASE WHEN d.outcome = 'Approved' THEN 1 ELSE 0 END) / COUNT(*), 1) AS
approval_pct
```

```
  FROM applicants a
  JOIN decisions d ON a.id = d.id
  GROUP BY a.a9
  ORDER BY approval_pct DESC")


# Q4: approval rate by a1 (we will reuse this grouping for the fairness audit in fairness audit)
dbGetQuery(con, "
  SELECT a.a1,
         COUNT(*) AS n,
         ROUND(100.0 * SUM(CASE WHEN d.outcome = 'Approved' THEN 1 ELSE 0 END) / COUNT(*), 1) AS
approval_pct
  FROM applicants a
  JOIN decisions d ON a.id = d.id
  WHERE a.a1 IS NOT NULL
  GROUP BY a.a1")


# Q5: average of two numeric columns by outcome
dbGetQuery(con, "
  SELECT d.outcome,
         ROUND(AVG(a.a2), 2)  AS avg_a2,
         ROUND(AVG(a.a15), 1) AS avg_a15
  FROM applicants a
  JOIN decisions d ON a.id = d.id
  GROUP BY d.outcome")


# Q6: missing values per column, counted in SQL
dbGetQuery(con, "
  SELECT SUM(CASE WHEN a1  IS NULL THEN 1 ELSE 0 END) AS a1_missing,
         SUM(CASE WHEN a2  IS NULL THEN 1 ELSE 0 END) AS a2_missing,
         SUM(CASE WHEN a4  IS NULL THEN 1 ELSE 0 END) AS a4_missing,
         SUM(CASE WHEN a5  IS NULL THEN 1 ELSE 0 END) AS a5_missing,
         SUM(CASE WHEN a6  IS NULL THEN 1 ELSE 0 END) AS a6_missing,
         SUM(CASE WHEN a7  IS NULL THEN 1 ELSE 0 END) AS a7_missing,
         SUM(CASE WHEN a14 IS NULL THEN 1 ELSE 0 END) AS a14_missing
  FROM applicants")


# Q7: filtered query with a safely-inserted value (works in SQLite and PostgreSQL)
q7 <- sqlInterpolate(con, "
  SELECT a.id, a.a2, a.a15, d.outcome
  FROM applicants a
  JOIN decisions d ON a.id = d.id
  WHERE a.a15 > ?min_a15
  ORDER BY a.a15 DESC
  LIMIT 10", min_a15 = 1000)
dbGetQuery(con, q7)


dbDisconnect(con)
```

da2_02_features.R

```
# ============================================================
# modeling - Part 2: Feature engineering & feature selection
# Reads the data back from the database (not from a CSV),
# splits it, imputes, engineers new features, and selects the best ones.
# ============================================================


library(tidyverse)
library(caret)
library(ranger)
library(DBI)
source("da2_db_config.R")


# ---- 1. Retrieve the modelling table from the database ----
con <- get_con()
df <- dbGetQuery(con, "
  SELECT a.*, d.outcome
  FROM applicants a
  JOIN decisions d ON a.id = d.id
  ORDER BY a.id")
dbDisconnect(con)


names(df) <- toupper(names(df))
df <- df %>% rename(id = ID, outcome = OUTCOME)


cat_cols <- c("A1", "A4", "A5", "A6", "A7", "A9", "A10", "A12", "A13")
num_cols <- c("A2", "A3", "A8", "A11", "A14", "A15")


df$outcome <- factor(df$outcome, levels = c("Approved", "Rejected"))
dim(df)   # expect 690 rows, 17 columns (id, A1-A15, outcome)


# ---- 2. Train / test split (80/20, stratified on the outcome) ----
# We split BEFORE imputing so the test set never influences the training steps.
set.seed(42)
```

```
train_idx <- createDataPartition(df$outcome, p = 0.8, list = FALSE)
train <- df[train_idx, ]
test  <- df[-train_idx, ]
table(train$outcome); table(test$outcome)


# ---- 3. Impute missing values using TRAINING-set statistics only ----
mode_of <- function(x) {
  x <- x[!is.na(x)]
  names(which.max(table(x)))
}
medians <- sapply(train[num_cols], median, na.rm = TRUE)
modes   <- sapply(train[cat_cols], mode_of)


impute <- function(d) {
  for (v in num_cols) d[[v]][is.na(d[[v]])] <- medians[[v]]
  for (v in cat_cols) d[[v]][is.na(d[[v]])] <- modes[[v]]
  d
}
train <- impute(train)
test  <- impute(test)


# Turn categorical columns into factors with the same levels in train and test
all_levels <- lapply(df[cat_cols], function(x) sort(unique(na.omit(x))))
to_factor <- function(d) {
  for (v in cat_cols) d[[v]] <- factor(d[[v]], levels = all_levels[[v]])
  d
}
train <- to_factor(train)
test  <- to_factor(test)


colSums(is.na(train)); colSums(is.na(test))   # all zeros expected


# ---- 4. FEATURE ENGINEERING ----
# Age-like column A2 -> 4 groups using TRAINING quartiles (used later for the fairness audit)
age_breaks <- quantile(train$A2, probs = seq(0, 1, 0.25))
age_breaks[1] <- -Inf
age_breaks[5] <- Inf


engineer <- function(d) {
  d %>% mutate(
    # (a) log transforms for the right-skewed numeric columns
    A3_log  = log1p(A3),
    A8_log  = log1p(A8),
    A11_log = log1p(A11),
    A14_log = log1p(A14),
    A15_log = log1p(A15),
    # (b) "is it non-zero?" flags, because A11 and A15 are zero for many applicants
    A11_pos = factor(ifelse(A11 > 0, "yes", "no"), levels = c("no", "yes")),
    A15_pos = factor(ifelse(A15 > 0, "yes", "no"), levels = c("no", "yes")),
    # (c) a ratio feature
    A8_per_A2 = A8 / A2,
    # (d) interaction flag: both binary flags A9 and A10 are "t"
    A9_and_A10 = factor(ifelse(A9 == "t" & A10 == "t", "yes", "no"), levels = c("no", "yes")),
    # (e) group column for fairness analysis (NOT used as a model feature)
    A2_group = cut(A2, breaks = age_breaks, labels = c("Q1", "Q2", "Q3", "Q4"))
  )
}
train <- engineer(train)
test  <- engineer(test)


# ---- 5. FEATURE SELECTION ----
candidate_features <- setdiff(names(train), c("id", "outcome", "A2_group"))


# Step 1: remove near-zero-variance features
nzv <- nearZeroVar(train[, candidate_features], saveMetrics = TRUE)
print(nzv[nzv$nzv, ])
candidate_features <- setdiff(candidate_features, rownames(nzv)[nzv$nzv])


# Step 2: remove one of any pair of highly correlated numeric features (|r| > 0.90)
num_feats <- candidate_features[sapply(train[, candidate_features], is.numeric)]
cor_mat   <- cor(train[, num_feats])
drop_corr <- findCorrelation(cor_mat, cutoff = 0.90, names = TRUE)
cat("Dropped for high correlation:", drop_corr, "\n")
candidate_features <- setdiff(candidate_features, drop_corr)


# Step 3: rank what is left with random-forest permutation importance
rf_data <- train[, c("outcome", candidate_features)]
set.seed(42)
rf  <- ranger(outcome ~ ., data = rf_data, num.trees = 500,
              importance = "permutation")
imp <- sort(rf$variable.importance, decreasing = TRUE)
print(round(imp, 4))
```

```
# Keep features with positive importance (always keep at least the top 5)
selected <- names(imp)[imp > 0]
if (length(selected) < 5) selected <- names(imp)[1:5]
cat("Selected", length(selected), "of", length(candidate_features), "features:\n")
print(selected)


# Plot of importances
imp_df <- data.frame(feature = names(imp), importance = as.numeric(imp)) %>%
  mutate(kept = feature %in% selected)
p <- ggplot(imp_df, aes(x = reorder(feature, importance), y = importance, fill = kept)) +
  geom_col() +
  coord_flip() +
  labs(title = "Feature importance (random forest, permutation)",
       x = "Feature", y = "Importance", fill = "Selected") +
  theme_minimal()
print(p)
ggsave("da2_feature_importance.png", p, width = 7, height = 5, dpi = 150)


# ---- 6. Save for the modelling script ----
write.csv(imp_df, "da2_feature_importance.csv", row.names = FALSE)
saveRDS(list(train = train, test = test,
             features = selected,
             all_features = candidate_features),
        "da2_data.rds")
```

da2_03_models.R

```
# modeling - Part 3: Train and compare 12 classification models

# comparison is fair. Results are saved to CSV and to the database.
# ============================================================


library(tidyverse)
library(caret)
library(pROC)
library(DBI)
source("da2_db_config.R")




# ---- 1. Load the prepared data ----
d     <- readRDS("da2_data.rds")
train <- d$train
test  <- d$test
feats <- d$features


train_df <- train[, c("outcome", feats)]
test_df  <- test[,  c("outcome", feats)]
test_y   <- test$outcome


# Baseline: accuracy of always predicting the most common class
baseline <- max(prop.table(table(test_y)))
cat("Majority-class baseline accuracy:", round(baseline, 3), "\n")


# ---- 2. Cross-validation setup (identical folds for all models) ----
set.seed(123)
folds <- createMultiFolds(train_df$outcome, k = 5, times = 2)


ctrl <- trainControl(
  method          = "repeatedcv",
  number          = 5,
  repeats         = 2,
  index           = folds,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,   # gives ROC (AUC), Sens, Spec
  savePredictions = "final"
)


# ---- 3. The 12 models ----









# Random forest grid: mtry stays well below the number of columns, because
# the "zv" step can drop all-zero dummy columns inside a cross-validation fold
```

```
p <- ncol(model.matrix(outcome ~ ., data = train_df)) - 1
rf_grid <- expand.grid(
  mtry          = unique(pmax(1, round(c(0.15, 0.3, 0.5) * p))),
  splitrule     = c("gini", "extratrees"),
  min.node.size = c(1, 5)
)


specs <- list(
  "Logistic Regression"    = list(method = "glm"),
  "Elastic Net Logistic"   = list(method = "glmnet", tuneLength = 5),
  "LDA"                    = list(method = "lda"),
  "KNN"                    = list(method = "knn", tuneLength = 10),
  "Naive Bayes"            = list(method = "naive_bayes"),
  "Decision Tree"          = list(method = "rpart", tuneLength = 10),
  "Bagged Trees"           = list(method = "treebag"),
  "Random Forest"          = list(method = "ranger", tuneGrid = rf_grid),


  "SVM Linear"             = list(method = "svmLinear", tuneLength = 4),
  "SVM RBF"                = list(method = "svmRadial", tuneLength = 5),
  "Neural Net MLP"         = list(method = "nnet", tuneLength = 4,
                                  trace = FALSE, MaxNWts = 5000, maxit = 300)
)

# ---- 4. Train every model (one failure will not stop the others) ----
models <- list()
for (nm in names(specs)) {
  message("Training: ", nm)
  set.seed(123)
  models[[nm]] <- tryCatch(
    do.call(caret::train, c(
      list(outcome ~ ., data = train_df, trControl = ctrl,
           metric = "ROC", preProcess = c("zv", "center", "scale")),
      specs[[nm]]
    )),
    error = function(e) {
      message("  FAILED: ", nm, " -> ", conditionMessage(e))
      NULL
    }
  )
}
models <- Filter(Negate(is.null), models)
cat("\nModels trained successfully:", length(models), "of", length(specs), "\n")


# ---- 5. Cross-validation results ----
cv_tbl <- data.frame(
  Model  = names(models),
  CV_AUC = sapply(models, function(m) getTrainPerf(m)$TrainROC),
  row.names = NULL
)
print(cv_tbl[order(-cv_tbl$CV_AUC), ])


rs <- resamples(models)
print(bwplot(rs, metric = "ROC", main = "Cross-validated AUC by model"))


# ---- 6. Evaluation on the held-out test set ----
test_df_x <- test_df[, feats, drop = FALSE]


eval_model <- function(m, nm) {
  pred <- predict(m, test_df_x)
  prob <- predict(m, test_df_x, type = "prob")[, "Approved"]
  cm   <- confusionMatrix(pred, test_y, positive = "Approved")
  roc_obj <- roc(response = test_y, predictor = prob,
                 levels = c("Rejected", "Approved"), direction = "<", quiet = TRUE)
  data.frame(
    Model     = nm,
    Accuracy  = as.numeric(cm$overall["Accuracy"]),
    Precision = as.numeric(cm$byClass["Precision"]),
    Recall    = as.numeric(cm$byClass["Recall"]),
    F1        = as.numeric(cm$byClass["F1"]),
    AUC       = as.numeric(auc(roc_obj))
  )
}

results <- bind_rows(lapply(names(models), function(nm) eval_model(models[[nm]], nm))) %>%
  left_join(cv_tbl, by = "Model") %>%
  arrange(desc(AUC))
results[, -1] <- round(results[, -1], 3)
print(results)


write.csv(results, "da2_model_results.csv", row.names = FALSE)


# Comparison chart
plot_df <- results %>%
```

```
  select(Model, Accuracy, F1, AUC) %>%
  pivot_longer(-Model, names_to = "Metric", values_to = "Value")
p <- ggplot(plot_df, aes(x = reorder(Model, Value), y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(title = "Test-set performance by model", x = NULL, y = "Score") +
  theme_minimal()
print(p)
ggsave("da2_model_comparison.png", p, width = 8, height = 6, dpi = 150)


# ---- 7. Save test predictions (needed for the fairness audit in fairness audit) ----
pred_tbl <- data.frame(
  id       = test$id,
  true     = test_y,
  A1       = test$A1,
  A2_group = test$A2_group
)
for (nm in names(models)) {
  key <- make.names(nm)
  pred_tbl[[paste0(key, "_pred")]] <- predict(models[[nm]], test_df_x)
  pred_tbl[[paste0(key, "_prob")]] <- predict(models[[nm]], test_df_x, type = "prob")[, "Approved"]
}
write.csv(pred_tbl, "da2_test_predictions.csv", row.names = FALSE)


# ---- 8. Store the results table in the database and read it back with SQL ----
con <- get_con()
res_db <- results
names(res_db) <- tolower(names(res_db))
dbWriteTable(con, "model_results", res_db, overwrite = TRUE)


dbGetQuery(con, "
  SELECT model, accuracy, f1, auc
  FROM model_results
  ORDER BY auc DESC
  LIMIT 5")


dbDisconnect(con)
```

Appendix B · Fairness Audit & Dashboard Code

Run order: `da3_01_fairness.R`, then `da3_02_without_A1.R`, then the dashboard with `shiny::runApp("da3_dashboard")`. `da3_helpers.R` is loaded by both scripts.

da3_helpers.R

```
# ============================================================
# fairness audit - Helper functions shared by the fairness scripts
# ============================================================
# Conventions used everywhere:
#   - "Approved" is the positive class
#   - pos = logical vector, TRUE where the applicant truly was approved
#   - pp  = logical vector, TRUE where the model predicted "Approved"
#   - g   = integer group codes (1, 2, ...) for the group column being audited


# ---- Per-group numbers (used for the reported tables) ----
group_stats <- function(truth, predicted) {
  pos <- truth == "Approved"
  pp  <- predicted == "Approved"
  data.frame(
    n              = length(truth),
    n_true_approved = sum(pos),
    n_true_rejected = sum(!pos),
    approval_rate   = mean(pp),                                  # share the model approves
    tpr             = if (any(pos))  mean(pp[pos])  else NA_real_,  # approved among truly approved
    fpr             = if (any(!pos)) mean(pp[!pos]) else NA_real_,  # approved among truly rejected
    accuracy        = mean(pp == pos)
  )
}

group_table <- function(truth, predicted, group) {
  out <- lapply(split(seq_along(truth), group), function(idx) {
    cbind(group = as.character(group[idx[1]]),
          group_stats(truth[idx], predicted[idx]),
          stringsAsFactors = FALSE)
  })
  out <- do.call(rbind, out)
  rownames(out) <- NULL
  out
}

range_gap <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) < 2) NA_real_ else max(x) - min(x)
}

# ---- Fast version used inside bootstrap / permutation loops ----
# Returns the four fairness numbers for one model and one group column:
#   dp_gap    = largest difference in approval rate between groups (demographic parity)
#   eo_gap    = largest difference in true-positive rate (equal opportunity)
#   eodds_gap = larger of the TPR gap and the false-positive-rate gap (equalized odds)
#   di_ratio  = lowest approval rate / highest approval rate (the "80% rule" ratio)
gaps_fast <- function(pos, pp, g) {
  k     <- max(g)
  n_g   <- tabulate(g, k)
  n_pos <- tabulate(g[pos], k)
  n_neg <- tabulate(g[!pos], k)
  app   <- ifelse(n_g   > 0, tabulate(g[pp], k)        / n_g,   NA_real_)
  tpr   <- ifelse(n_pos > 0, tabulate(g[pp & pos], k)  / n_pos, NA_real_)
  fpr   <- ifelse(n_neg > 0, tabulate(g[pp & !pos], k) / n_neg, NA_real_)
  dp  <- range_gap(app)
  eo  <- range_gap(tpr)
  fp  <- range_gap(fpr)
  both <- c(eo, fp)
  mx  <- suppressWarnings(max(app, na.rm = TRUE))
  mn  <- suppressWarnings(min(app, na.rm = TRUE))
  c(dp_gap    = dp,
    eo_gap    = eo,
    eodds_gap = if (all(is.na(both))) NA_real_ else max(both, na.rm = TRUE),
    di_ratio  = if (is.finite(mx) && mx > 0) mn / mx else NA_real_)
}

# ---- AUC without pROC (fast enough for thousands of bootstrap resamples) ----
# Equivalent to the Mann-Whitney U statistic; ties get average ranks.
fast_auc <- function(pos, prob) {
  n1 <- sum(pos)
  n0 <- sum(!pos)
  if (n1 == 0 || n0 == 0) return(NA_real_)
  r <- rank(prob)
  (sum(r[pos]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}
```

```
# 95% percentile interval
ci95 <- function(x) unname(quantile(x, c(0.025, 0.975), na.rm = TRUE))


# Permutation p-value: how often does a gap this large appear by chance
# when group labels are shuffled? (add-one correction avoids p = 0)
perm_p <- function(null_values, observed) {
  if (is.na(observed)) return(NA_real_)
  v <- null_values[!is.na(null_values)]
  (sum(v >= observed) + 1) / (length(v) + 1)
}
```

da3_01_fairness.R

```
# ============================================================
# fairness audit - Part 1: Fairness audit and statistical comparison
# Input : da2_test_predictions.csv, da2_model_results.csv  (from modeling)
# Output: da3_*.csv tables, da3_*.png plots, and a data folder for the dashboard
# ============================================================


library(dplyr)
library(tidyr)
library(ggplot2)
library(pROC)
source("da3_helpers.R")


B_BOOT <- 1000   # bootstrap resamples (confidence intervals)
B_PERM <- 2000   # label shuffles ("is this gap bigger than chance?")
set.seed(2026)


# ---- 1. Load the modeling predictions ----
pred <- read.csv("da2_test_predictions.csv", stringsAsFactors = FALSE)
res  <- read.csv("da2_model_results.csv",    stringsAsFactors = FALSE)


truth <- as.character(pred$true)
pos   <- truth == "Approved"
n     <- nrow(pred)
cat("Test applicants:", n, "| truly approved:", sum(pos), "\n")


# Model keys come from the column names, e.g. "Random.Forest_pred"
keys     <- sub("_pred$", "", grep("_pred$", names(pred), value = TRUE))
name_map <- setNames(res$Model, make.names(res$Model))
display  <- unname(ifelse(keys %in% names(name_map), name_map[keys],
                          gsub(".", " ", keys, fixed = TRUE)))
cat("Models found:", length(keys), "\n")


# Groups being audited. They are ANONYMOUS groups (the dataset hides the real meaning).
group_vars <- c("A1", "A2_group")
for (gv in group_vars) pred[[gv]] <- as.character(pred[[gv]])


# ---- 2. Base rates: how the TRUE labels differ between groups ----
# If the real approval rates already differ, some model gap is expected.
base_rates <- bind_rows(lapply(group_vars, function(gv) {
  data.frame(group_var = gv, group = pred[[gv]], approved = pos) %>%
    group_by(group_var, group) %>%
    summarise(n = n(), true_approval_rate = mean(approved), .groups = "drop")
}))


# ---- 3. Fairness metrics for every model and group column ----
model_rows <- list()
group_rows <- list()
fair_rows  <- list()


for (i in seq_along(keys)) {
  key       <- keys[i]
  mname     <- display[i]
  predicted <- as.character(pred[[paste0(key, "_pred")]])
  prob      <- pred[[paste0(key, "_prob")]]
  pp        <- predicted == "Approved"
  message("Auditing: ", mname)


  # 3a. Overall accuracy and AUC with bootstrap confidence intervals
  acc <- mean(pp == pos)
  auc <- fast_auc(pos, prob)
  bs  <- replicate(B_BOOT, {
    idx <- sample.int(n, replace = TRUE)
    c(mean(pp[idx] == pos[idx]), fast_auc(pos[idx], prob[idx]))
  })
  model_rows[[i]] <- data.frame(
    model = mname,
    accuracy = acc, acc_lo = ci95(bs[1, ])[1], acc_hi = ci95(bs[1, ])[2],
    auc = auc,      auc_lo = ci95(bs[2, ])[1], auc_hi = ci95(bs[2, ])[2],
    cv_auc = res$CV_AUC[match(mname, res$Model)]
  )
```

```
  # 3b. Fairness gaps per group column
  for (gv in group_vars) {
    g   <- as.integer(factor(pred[[gv]]))
    gt  <- group_table(truth, predicted, pred[[gv]])
    group_rows[[length(group_rows) + 1]] <- cbind(model = mname, group_var = gv, gt,
                                                  stringsAsFactors = FALSE)


    obs  <- gaps_fast(pos, pp, g)
    boot <- replicate(B_BOOT, {
      idx <- sample.int(n, replace = TRUE)
      gaps_fast(pos[idx], pp[idx], g[idx])
    })
    perm <- replicate(B_PERM, gaps_fast(pos, pp, sample(g)))   # labels shuffled = no real group effect


    fair_rows[[length(fair_rows) + 1]] <- data.frame(
      model = mname, group_var = gv, n_groups = max(g),
      dp_gap = obs["dp_gap"],
      dp_lo = ci95(boot["dp_gap", ])[1], dp_hi = ci95(boot["dp_gap", ])[2],
      dp_chance = mean(perm["dp_gap", ], na.rm = TRUE),
      dp_p = perm_p(perm["dp_gap", ], obs["dp_gap"]),
      eo_gap = obs["eo_gap"],
      eo_lo = ci95(boot["eo_gap", ])[1], eo_hi = ci95(boot["eo_gap", ])[2],
      eo_chance = mean(perm["eo_gap", ], na.rm = TRUE),
      eo_p = perm_p(perm["eo_gap", ], obs["eo_gap"]),
      eodds_gap = obs["eodds_gap"],
      di_ratio = obs["di_ratio"],
      row.names = NULL
    )
  }
}

model_summary <- bind_rows(model_rows)
group_metrics <- bind_rows(group_rows)
fair          <- bind_rows(fair_rows)

# Holm correction: we run many tests, so p-values are adjusted for that
fair$dp_p_adj <- p.adjust(fair$dp_p, method = "holm")
fair$eo_p_adj <- p.adjust(fair$eo_p, method = "holm")


# ---- 4. Is the best model really better than the others? ----
# Reference = highest test AUC. Two paired tests on the SAME test applicants:
#   DeLong test       -> are the AUCs different?
#   Exact McNemar test-> do the two models get different applicants right?
ref_i    <- which.max(model_summary$auc)
resp     <- factor(ifelse(pos, "Approved", "Rejected"), levels = c("Rejected", "Approved"))
roc_list <- lapply(keys, function(k)
  roc(response = resp, predictor = pred[[paste0(k, "_prob")]], quiet = TRUE,
      levels = c("Rejected", "Approved"), direction = "<"))
pp_ref   <- as.character(pred[[paste0(keys[ref_i], "_pred")]]) == "Approved"
ok_ref   <- pp_ref == pos


test_rows <- list()
for (j in seq_along(keys)) {
  if (j == ref_i) next
  ok_j <- (as.character(pred[[paste0(keys[j], "_pred")]]) == "Approved") == pos
  n_ref_only   <- sum(ok_ref & !ok_j)     # reference right, other wrong
  n_other_only <- sum(!ok_ref & ok_j)     # other right, reference wrong
  p_de <- tryCatch(roc.test(roc_list[[ref_i]], roc_list[[j]],
                            method = "delong", paired = TRUE)$p.value,
                   error = function(e) NA_real_)
  p_mc <- if (n_ref_only + n_other_only == 0) 1 else
    binom.test(n_ref_only, n_ref_only + n_other_only, 0.5)$p.value
  test_rows[[length(test_rows) + 1]] <- data.frame(
    ref_model = display[ref_i], model = display[j],
    auc = model_summary$auc[j],
    auc_diff = model_summary$auc[ref_i] - model_summary$auc[j],
    delong_p = p_de,
    ref_only_correct = n_ref_only, model_only_correct = n_other_only,
    mcnemar_exact_p = p_mc
  )
}
model_tests <- bind_rows(test_rows)
model_tests$delong_p_adj  <- p.adjust(model_tests$delong_p, method = "holm")
model_tests$mcnemar_p_adj <- p.adjust(model_tests$mcnemar_exact_p, method = "holm")


# ---- 5. Save tables ----
write.csv(model_summary, "da3_model_summary.csv",   row.names = FALSE)
write.csv(group_metrics, "da3_group_metrics.csv",   row.names = FALSE)
write.csv(fair,          "da3_fairness_summary.csv", row.names = FALSE)
write.csv(model_tests,   "da3_model_tests.csv",     row.names = FALSE)
write.csv(base_rates,    "da3_base_rates.csv",      row.names = FALSE)


# ---- 6. Plots ----
```

```
gap_long <- bind_rows(
  fair %>% transmute(model, group_var, metric = "Approval-rate gap (demographic parity)",
                     gap = dp_gap, lo = dp_lo, hi = dp_hi),
  fair %>% transmute(model, group_var, metric = "Equal opportunity gap (TPR)",
                     gap = eo_gap, lo = eo_lo, hi = eo_hi)
)
p1 <- ggplot(gap_long, aes(x = reorder(model, gap), y = gap, ymin = lo, ymax = hi, colour = metric)) +
  geom_pointrange(position = position_dodge(width = 0.6), size = 0.35) +
  coord_flip() +
  facet_wrap(~ group_var, labeller = labeller(group_var = c(A1 = "Groups by A1", A2_group = "Groups by A2
quartile"))) +
  labs(title = "Fairness gaps by model (95% bootstrap interval)",
       x = NULL, y = "Gap (0 = groups treated equally)", colour = NULL) +
  theme_minimal() + theme(legend.position = "bottom")
ggsave("da3_fairness_gaps.png", p1, width = 10, height = 6, dpi = 150)


trade <- fair %>% left_join(model_summary, by = "model")
p2 <- ggplot(trade, aes(x = eo_gap, y = accuracy)) +
  geom_point(size = 2.5, colour = "#1F4E78") +
  geom_text(aes(label = model), size = 2.8, vjust = -0.8, check_overlap = TRUE) +
  facet_wrap(~ group_var, labeller = labeller(group_var = c(A1 = "Groups by A1", A2_group = "Groups by A2
quartile"))) +
  labs(title = "Accuracy vs fairness trade-off",
       x = "Equal opportunity gap (lower is fairer)", y = "Test accuracy") +
  theme_minimal()
ggsave("da3_tradeoff.png", p2, width = 10, height = 5.5, dpi = 150)


# ---- 7. Copy the tables into the dashboard folder ----
dir.create("da3_dashboard/data", recursive = TRUE, showWarnings = FALSE)
file.copy(c("da3_model_summary.csv", "da3_group_metrics.csv", "da3_fairness_summary.csv",
            "da3_model_tests.csv", "da3_base_rates.csv"),
          "da3_dashboard/data", overwrite = TRUE)


# ---- 8. Quick look at the results ----
cat("\n--- Base rates (true labels) ---\n");           print(base_rates)
cat("\n--- Model summary ---\n");                      print(model_summary %>%
mutate(across(where(is.numeric), ~ round(.x, 3))))
cat("\n--- Fairness summary ---\n");                   print(fair %>% mutate(across(where(is.numeric), ~
round(.x, 3))))
cat("\n--- Best model vs the rest ---\n");             print(model_tests %>%
mutate(across(where(is.numeric), ~ round(.x, 3))))
```

da3_02_without_A1.R

```
# ============================================================
# fairness audit - Part 2: Does removing A1 make the models fairer?
# ("Fairness through unawareness" check)
# Retrains three fast models with and without A1 on the same folds, then
# compares accuracy, fairness gaps, and how many predictions change.
# Input : da2_data.rds (from modeling)
# Output: da3_unawareness.csv, da3_unawareness.png
# ============================================================


library(caret)
library(dplyr)
library(ggplot2)
source("da3_helpers.R")


d     <- readRDS("da2_data.rds")
trn   <- d$train
tst   <- d$test
feats <- d$features
if (!"A1" %in% feats) message("A1 was not among the selected features, so removing it changes nothing.")


variants <- list("With A1" = feats, "Without A1" = setdiff(feats, "A1"))


truth <- as.character(tst$outcome)
pos   <- truth == "Approved"
group_vars <- c("A1", "A2_group")


# Same cross-validation folds for every model and variant
set.seed(123)
folds <- createMultiFolds(trn$outcome, k = 5, times = 2)
ctrl  <- trainControl(method = "repeatedcv", number = 5, repeats = 2, index = folds,
                      classProbs = TRUE, summaryFunction = twoClassSummary)


rows  <- list()
preds <- list()


for (v in names(variants)) {
  train_df <- trn[, c("outcome", variants[[v]])]
  test_x   <- tst[, variants[[v]], drop = FALSE]
```

```
  # random-forest grid kept below the column count for stability
  p <- ncol(model.matrix(outcome ~ ., data = train_df)) - 1
  rf_grid <- expand.grid(mtry = unique(pmax(1, round(c(0.15, 0.3, 0.5) * p))),
                         splitrule = c("gini", "extratrees"), min.node.size = c(1, 5))

  specs <- list(
    "Logistic Regression"  = list(method = "glm"),
    "Elastic Net Logistic" = list(method = "glmnet", tuneLength = 5),
    "Random Forest"        = list(method = "ranger", tuneGrid = rf_grid)
  )

  for (m in names(specs)) {
    message("Training: ", m, " (", v, ")")
    set.seed(123)
    fit <- do.call(caret::train, c(
      list(outcome ~ ., data = train_df, trControl = ctrl, metric = "ROC",
           preProcess = c("zv", "center", "scale")),
      specs[[m]]))
    cls  <- as.character(predict(fit, test_x))
    prob <- predict(fit, test_x, type = "prob")[, "Approved"]
    preds[[paste(m, v)]] <- cls
    pp <- cls == "Approved"

    for (gv in group_vars) {
      g <- as.integer(factor(as.character(tst[[gv]])))
      gg <- gaps_fast(pos, pp, g)
      rows[[length(rows) + 1]] <- data.frame(
        model = m, variant = v, group_var = gv,
        accuracy = mean(pp == pos), auc = fast_auc(pos, prob),
        dp_gap = gg["dp_gap"], eo_gap = gg["eo_gap"], row.names = NULL)
    }
  }
}

out <- bind_rows(rows)

# Share of test applicants whose predicted class changes when A1 is removed
changed <- sapply(c("Logistic Regression", "Elastic Net Logistic", "Random Forest"), function(m)
  mean(preds[[paste(m, "With A1")]] != preds[[paste(m, "Without A1")]]))
out$pct_predictions_changed <- round(100 * changed[out$model], 1)

write.csv(out, "da3_unawareness.csv", row.names = FALSE)
print(out %>% mutate(across(where(is.numeric), ~ round(.x, 3))))

long <- out %>%
  tidyr::pivot_longer(c(dp_gap, eo_gap), names_to = "metric", values_to = "gap") %>%
  mutate(metric = recode(metric, dp_gap = "Approval-rate gap", eo_gap = "Equal opportunity gap"))
p <- ggplot(long, aes(x = model, y = gap, fill = variant)) +
  geom_col(position = "dodge") +
  facet_grid(metric ~ group_var) +
  labs(title = "Fairness gaps with and without A1", x = NULL, y = "Gap", fill = NULL) +
  theme_minimal() + theme(axis.text.x = element_text(angle = 20, hjust = 1))
ggsave("da3_unawareness.png", p, width = 9, height = 6, dpi = 150)
```

da3_dashboard/app.R

```
# ============================================================
# Interactive dashboard (R Shiny + Plotly)
# Credit Card Approval: Model Performance & Fairness Audit
# Run with: shiny::runApp("da3_dashboard") after running the fairness analysis
# ============================================================

library(shiny)
library(plotly)
library(dplyr)

# ---- Load saved analysis results ----
data_dir <- if (dir.exists("data")) "data" else "."
rd <- function(f) read.csv(file.path(data_dir, f), stringsAsFactors = FALSE)

model_summary <- rd("da3_model_summary.csv")
group_metrics <- rd("da3_group_metrics.csv")
fair          <- rd("da3_fairness_summary.csv")
tests         <- rd("da3_model_tests.csv")
base_rates    <- rd("da3_base_rates.csv")

group_labels <- c(A1 = "A1 (2 groups)", A2_group = "A2 quartile (4 groups)")
model_names  <- model_summary$model[order(-model_summary$auc)]

fmt  <- function(x, d = 3) formatC(x, format = "f", digits = d)
pfmt <- function(p) ifelse(is.na(p), "NA", ifelse(p < 0.001, "< 0.001", fmt(p, 3)))
```

```
tile <- function(title, value, note = NULL) {
  div(class = "well", style = "text-align:center; padding:10px;",
      h5(title, style = "margin-top:0; color:#555;"),
      h3(value, style = "margin:4px 0; color:#1F4E78;"),
      if (!is.null(note)) p(note, style = "font-size:12px; color:#777; margin-bottom:0;"))
}

# ============================== UI ==============================
ui <- navbarPage(
  "Credit Approval: Model Performance & Fairness Audit",


  # ---------- 1. Overview ----------
  tabPanel("Overview",
    fluidRow(column(12,
      h4("What this dashboard shows"),
      p("Twelve classification models were trained on the UCI Credit Approval data (690 applicants) and",
        "tested on 137 applicants they had never seen. This dashboard compares their accuracy and checks",
        "whether each model treats applicant groups equally."),
      p(strong("Groups are anonymous."), "The dataset hides what each column means, so 'A1' and 'A2
quartile'",
        "are just two ways to split applicants into groups. They are not confirmed gender or age groups.")
    )),
    fluidRow(
      column(7, h4("AUC by model (95% bootstrap interval)"), plotlyOutput("auc_plot", height = "420px")),
      column(5, h4("Test-set metrics"), tableOutput("model_table"))
    )
  ),


  # ---------- 2. Fairness by model ----------
  tabPanel("Fairness by model",
    sidebarLayout(
      sidebarPanel(width = 3,
        selectInput("model", "Model", choices = model_names, selected = model_names[1]),
        radioButtons("gvar", "Group column", choices = setNames(names(group_labels), group_labels)),
        helpText("A gap of 0 means the groups are treated equally. The 'chance' value is the gap you
would",
                 "see from random variation alone with groups this small.")
      ),
      mainPanel(width = 9,
        uiOutput("tiles"),
        fluidRow(
          column(7, plotlyOutput("group_plot", height = "380px")),
          column(5, h5("Per-group numbers"), tableOutput("group_table"))
        )
      )
    )
  ),


  # ---------- 3. Trade-off ----------
  tabPanel("Accuracy vs fairness",
    sidebarLayout(
      sidebarPanel(width = 3,
        radioButtons("gvar2", "Group column", choices = setNames(names(group_labels), group_labels)),
        radioButtons("gap_type", "Fairness gap",
                     choices = c("Equal opportunity gap" = "eo", "Approval-rate gap" = "dp")),
        radioButtons("perf", "Performance measure", choices = c("Accuracy" = "accuracy", "AUC" = "auc")),
        helpText("Models toward the top-left are both accurate and fair. Hover over a point to see the
model.",
                 "Horizontal bars show the 95% bootstrap interval of the gap.")
      ),
      mainPanel(width = 9, plotlyOutput("trade_plot", height = "520px"))
    )
  ),


  # ---------- 4. Statistical tests ----------
  tabPanel("Is the best model really better?",
    fluidRow(column(12,
      uiOutput("test_intro"),
      tableOutput("test_table"),
      helpText("DeLong test compares AUCs; the exact McNemar test compares which applicants each model
gets",
               "right. p-values are Holm-adjusted for the number of comparisons. A large adjusted p-value
means",
               "the data cannot separate the two models.")
    ))
  ),


  # ---------- 5. Notes ----------
  tabPanel("Base rates & limitations",
    fluidRow(
      column(5,
        h4("True approval rate by group (test set)"),
        tableOutput("base_table"),
        helpText("If the real approval rates already differ between groups, some gap in model decisions
is",
```

```
                "expected. Equal opportunity compares only applicants who truly were approved.")
      ),
      column(7,
        h4("Limitations"),
        tags$ul(
          tags$li("Column meanings are hidden, so the groups are anonymous, not confirmed demographic
groups."),
          tags$li("The labels are the bank's past decisions, not verified defaults, so any historical bias
is in the labels."),
          tags$li("The test set has 137 applicants. Group sizes are small, so gaps are noisy; use the
intervals and chance values."),
          tags$li("Intervals for a gap are skewed because a gap cannot be negative; the chance comparison



      )
    )
  )
)

# ============================ Server ============================
server <- function(input, output, session) {

  # ---- Overview ----
  output$auc_plot <- renderPlotly({
    d <- model_summary %>% mutate(model = factor(model, levels = rev(model[order(auc)])))
    plot_ly(d, x = ~auc, y = ~model, type = "scatter", mode = "markers",
            marker = list(color = "#1F4E78", size = 9),
            error_x = list(type = "data", symmetric = FALSE,
                           array = ~(auc_hi - auc), arrayminus = ~(auc - auc_lo), color = "#7a9cc0"),
            text = ~paste0(model, "<br>AUC ", fmt(auc), " (", fmt(auc_lo), " to ", fmt(auc_hi), ")"),
            hoverinfo = "text") %>%
      layout(xaxis = list(title = "Test AUC"), yaxis = list(title = ""),
             margin = list(l = 150))
  })

  output$model_table <- renderTable({
    model_summary %>% arrange(desc(auc)) %>%
      transmute(Model = model, Accuracy = fmt(accuracy), AUC = fmt(auc), `CV AUC` = fmt(cv_auc))
  }, striped = TRUE, spacing = "s")


  # ---- Fairness by model ----
  sel_fair <- reactive(fair %>% filter(model == input$model, group_var == input$gvar))
  sel_group <- reactive(group_metrics %>% filter(model == input$model, group_var == input$gvar))


  output$tiles <- renderUI({
    f <- sel_fair(); req(nrow(f) == 1)
    verdict <- function(f) {
      flags <- c(if (!is.na(f$dp_p) && f$dp_p < 0.05) "approval-rate gap",
                 if (!is.na(f$eo_p) && f$eo_p < 0.05) "equal opportunity gap")
      if (length(flags) == 0) "within chance variation"
      else paste("larger than chance:", paste(flags, collapse = " and "))
    }
    fluidRow(
      column(3, tile("Approval-rate gap", fmt(f$dp_gap),
                     paste0("95% interval ", fmt(f$dp_lo), " to ", fmt(f$dp_hi)))),
      column(3, tile("Equal opportunity gap", fmt(f$eo_gap),
                     paste0("95% interval ", fmt(f$eo_lo), " to ", fmt(f$eo_hi)))),
      column(3, tile("Gap expected by chance", paste0(fmt(f$dp_chance, 2), " / ", fmt(f$eo_chance, 2)),
                     "approval-rate / equal opportunity")),
      column(3, tile("Verdict (unadjusted)", verdict(f),
                     paste0("p = ", pfmt(f$dp_p), " (approval), ", pfmt(f$eo_p), " (equal opp.)")))
    )
  })

  output$group_plot <- renderPlotly({
    g <- sel_group(); req(nrow(g) > 0)
    d <- bind_rows(
      g %>% transmute(group, metric = "Approval rate (all applicants)", value = approval_rate),
      g %>% transmute(group, metric = "True positive rate (truly approved)", value = tpr))
    plot_ly(d, x = ~group, y = ~value, color = ~metric, type = "bar",
            colors = c("#1F4E78", "#e08a1e"),
            text = ~fmt(value, 2), textposition = "outside") %>%
      layout(barmode = "group", yaxis = list(title = "Rate", range = c(0, 1.1)),
             xaxis = list(title = group_labels[[input$gvar]]),
             legend = list(orientation = "h", y = -0.2))
  })

  output$group_table <- renderTable({
    sel_group() %>%
      transmute(Group = group, N = n, `Truly approved` = n_true_approved,
                `Approval rate` = fmt(approval_rate, 2), TPR = fmt(tpr, 2),
                FPR = fmt(fpr, 2), Accuracy = fmt(accuracy, 2))
  }, striped = TRUE, spacing = "s")
```

```
  # ---- Trade-off ----
  output$trade_plot <- renderPlotly({
    f <- fair %>% filter(group_var == input$gvar2) %>% left_join(model_summary, by = "model")
    gap <- f[[paste0(input$gap_type, "_gap")]]
    lo  <- f[[paste0(input$gap_type, "_lo")]]
    hi  <- f[[paste0(input$gap_type, "_hi")]]
    perf <- f[[input$perf]]
    plot_ly(x = gap, y = perf, type = "scatter", mode = "markers",
            marker = list(color = "#1F4E78", size = 11),
            error_x = list(type = "data", symmetric = FALSE,
                           array = hi - gap, arrayminus = gap - lo, color = "#b7c7da"),
            hovertext = paste0(f$model, "<br>gap ", fmt(gap), "<br>", input$perf, " ", fmt(perf)),
            hoverinfo = "text") %>%
      layout(xaxis = list(title = if (input$gap_type == "eo") "Equal opportunity gap (lower is fairer)"
                                  else "Approval-rate gap (lower is more equal)"),
             yaxis = list(title = if (input$perf == "accuracy") "Test accuracy" else "Test AUC"),
             showlegend = FALSE)
  })

  # ---- Tests ----
  output$test_intro <- renderUI({
    p(strong("Reference model: ", tests$ref_model[1]), " (highest test AUC), compared with every other
model on the same 137 test applicants.")
  })
  output$test_table <- renderTable({
    tests %>% arrange(delong_p) %>%
      transmute(Model = model, AUC = fmt(auc), `AUC difference` = fmt(auc_diff),
                `DeLong p (adj.)` = pfmt(delong_p_adj),
                `Reference right, model wrong` = ref_only_correct,
                `Model right, reference wrong` = model_only_correct,
                `McNemar p (adj.)` = pfmt(mcnemar_p_adj))
  }, striped = TRUE, spacing = "s")

  # ---- Notes ----
  output$base_table <- renderTable({
    base_rates %>% transmute(`Group column` = group_var, Group = group, N = n,
                             `True approval rate` = fmt(true_approval_rate, 2))
  }, striped = TRUE, spacing = "s")
}

shinyApp(ui, server)
```